



Cairo Campus of



TRUSTRA

Where Business Meets Trust Era



Dr. Dalia Fadl



TA: Mohamed Osama

Name	ID
Jana Mostafa Ibrahim	202206365
Mina Romany Shawky	202206484
Ahmed Adel Mohamed	202206321
Menna Tarek Abdelhamid	202206231



2025/2026

CIC – CANADIAN INTERNATIONAL COLLEGE
Business Information Technology – Dual Program

First and foremost, we would like to express our deepest gratitude to our college, our respected professors, teaching assistants, and everyone who supported us throughout our academic journey. Their guidance, encouragement, and dedication have played a vital role in helping us reach this milestone.

We would like to extend our special thanks and sincere appreciation to **Dr. Dalia Fadi** for her continuous support, valuable feedback, timely approvals, and exceptional mentorship. Her guidance has been instrumental not only in the successful completion of this graduation project but also throughout our academic journey, where she has consistently inspired and supported us.

We would also like to express our heartfelt gratitude to **Dr. Mohamed Hassan** for his exceptional guidance and unwavering support throughout our academic journey. His mentorship, insightful advice, and continuous encouragement have greatly influenced both our academic and professional growth. Beyond the classroom, he inspired us to think critically, strive for excellence, and prepare for the challenges of the workplace. Words cannot fully express our appreciation or give him the recognition he truly deserves.

Our sincere thanks also go to our amazing Teaching Assistant, **Mohamed Osama**, who worked tirelessly alongside us from the beginning of this journey. His dedication, patience, and willingness to provide assistance whenever needed greatly contributed to the success of our project, and we are truly grateful for all his efforts.

We would also like to recognize **Teaching Assistant Ammar Yasser** for being one of the most supportive advisors and instructors throughout our journey. His guidance, encouragement, and commitment to helping us overcome challenges made a significant difference in our learning experience.

In addition, we extend our deepest appreciation to **Mohamed Khaled**, who taught us what it truly means to be Business Information Technology students. Through his guidance, he helped us develop the mindset and skills required to think and act like real developers and analysts. His support and mentorship have left a lasting impression on us and will continue to influence our future careers.

Finally, we extend our sincere appreciation to our families and friends for their continuous encouragement, understanding, and unwavering support throughout this journey. Their belief in us has been a constant source of motivation and strength, enabling us to overcome challenges and successfully complete this project.



We dedicate this work to our beloved families, whose unconditional love, endless sacrifices, patience, and constant encouragement have been the foundation of our success. Their unwavering belief in our abilities has inspired us to persevere through every challenge and celebrate every achievement.

We also dedicate this project to our friends, who stood by our side throughout this journey, offering their support, encouragement, and valuable insights whenever needed.

Finally, we dedicate this work to all business owners, aspiring entrepreneurs, future leaders, and innovators who strive to create meaningful change and make a positive impact on society. We hope that our efforts contribute, even in a small way, to inspiring continued innovation and progress.



The rapid spread of fake business news across digital platforms has become a significant challenge for entrepreneurs, investors, and business owners, as misleading information can negatively influence financial decisions, business strategies, and public trust. To address this issue, this project presents **Trustra**, an AI-powered mobile application that combines Big Data technologies and Deep Learning techniques to detect fake business news and provide intelligent business support.

The proposed system enables users to upload or enter business-related news articles and social media content, which are analyzed using a fine-tuned **DistilBERT** model based on Natural Language Processing (NLP). The model classifies the submitted content as either true or fake, helping users verify information before making critical business decisions.

Beyond fake news detection, Trustra integrates **Trusty**, an AI-powered business expert chatbot that serves as a virtual consultant for entrepreneurs and business owners. Trusty provides assistance in business planning, feasibility studies, SWOT analysis, marketing strategies, financial management, startup development, and other business-related activities, offering users accessible and intelligent guidance tailored to their needs.

To ensure a reliable information ecosystem, the application includes a trusted community feed that publishes verified business content while restricting user interactions to reactions only, reducing the spread of misinformation through comments and discussions. Also, users can privately save, or publicly repost news to their profile. The platform also supports secure user authentication, account management, and profile customization to provide a seamless user experience.

From the data engineering perspective, the project implements an end-to-end Big Data pipeline that ingests, processes, and transforms data into analytics-ready datasets stored in a Snowflake data warehouse. The processed data powers interactive Power BI dashboards, including business intelligence insights and market research findings collected through user surveys that helped validate the project's concept and identify user needs.

The complete solution is deployed as a mobile application backed by a Flask API, combining Big Data, Deep Learning, Natural Language Processing, Business Intelligence, and intelligent conversational AI into a unified platform. Trustra contributes to combating business misinformation while empowering entrepreneurs with trusted knowledge, data-driven insights, and expert assistance to support smarter business decisions.



Table 1: Data Engineering Tools.....13

Table 2: AI/DL Tools.....13

Table 3: Backend Tools.....14

Table 4: Frontend Tools.....15

Table 5: Model Architecture Overview.....60

Table 6: Compare DistilBERT.....60

Table 7: Dataset Statistics.....61

Table 8: Techniques used in Evaluation Metrics.....62

Table 9: Overfitting Prevention Techniques.....63

Table 10: Model Accuracy Results.....63

Table 11: Fake News Detection — Test Set Performance.....64

Table 12: Category Classification — Test Set Performance.....64

Table 13: Public Endpoints.....67

Table 14: Protected Endpoints.....68

Table 15: Rate Limiting Strategy.....69

Table 16: Token Architecture.....70

Table 17: Registration & Validation Rules.....70

Table 18: History Management.....75

Table 19: Errors Array.....76

Table 20: Future AI Improvements.....87

Figure 1: Trustra Dashboard.....	9
Figure 2: Trustra Dashboard-2.....	9
Figure 3: Trustra SWOT Analysis.....	12
Figure 4: Thesis Structure.....	15
Figure 5: Use case-1.....	27
Figure 6: Use case-2.....	28
Figure 7: Use case-3.....	29
Figure 8: Class Diagram.....	47
Figure 9: Activity Diagram.....	48
Figure 10: Sequence Diagram.....	49
Figure 11: Entity Relationship Diagram.....	50
Figure 12: Home screen/Community feed.....	53
Figure 13: Trusty Screen (Chatbot)	54
Figure 14: Sign-Up Screen.....	56
Figure 15: Sign-In Screen.....	57
Figure 16: Data Engineering Pipeline.....	58
Figure 17: Training and validation curves over 5 epochs — Total Loss (left), Fake Detection Accuracy (center), Category Accuracy (right)	65
Figure 18: Confusion matrices for Fake News Detection (left) and Category Classification (right) on the held-out test set.....	65
Figure 19: Code sample-1.....	78
Figure 20: Code sample-2.....	79
Figure 21: Code sample-3.....	79
Figure 22: Code sample-4.....	80
Figure 23: Code sample-5.....	80
Figure 24: Code sample-6.....	81
Figure 25: Code sample-7.....	81

Section	Page Number
ACKNOWLEDGEMENT	i
DEDICATION	ii
ABSTRACT	iii
LIST OF TABLES	vi
LIST OF FIGURES	viii
Chapter 1: Introduction	8
1.1 Overview	9
1.2 Executive Summary	9
1.3 Problem Definition	11
1.4 Objectives of the Project	11
1.5 Motivation of the Work	11
1.6 Used Tools	13
1.6.1 Data Engineering Tools	13
1.6.2 AI/DL Tools	13
1.6.3 Backend Tools	14
1.6.4 Frontend Tools	15
1.7 Thesis Structure	15
Chapter 2: State of the Art	16
2.1 Overview	17
2.2 Existing Projects	17
2.3 Comparative Analysis Summary	21
Chapter 3: System Analysis and Design	22
3.1 Overview	23
3.2 System Analysis	24
3.2.1 Project Requirements	23
3.2.2 Use Cases	29
3.2.3 Class Diagram	49
3.2.4 Activity Diagram	50
3.2.5 Sequence Diagram	51
3.3 System Design	52

3.3.1 Entity Relationship Diagram	52
3.3.2 Interface Outlines	53
Chapter 4: The Proposed Analytics Tier	57
4.1 Data Engineering	58
4.1.1 Data Engineering Pipeline Using Big Data	58
4.1.2 ETL Workflow	58
4.1.3 Scalability Discussion	59
4.2 AI/DL	59
4.2.1 Model Architecture	59
4.2.2 NLP Pipeline	60
4.2.3 Training and Testing	60
4.2.4 Evaluation Metrics	62
4.2.5 Accuracy Results	63
4.2.6 Confusion Matrix	64
4.2.7 Training Curves	65
4.2.8 Confusion Matrix	65
4.3 Backend API Implementation	66
4.3.1 Authentication System	66
4.3.2 Endpoint Architecture	67
4.3.3 News Verification Pipeline	68
4.3.4 Rate Limiting Strategy	69
4.3.5 Authentication System (Detail)	70
4.3.6 Integration with Frontend, Trusty and AI Model	72
4.3.7 Additional Technical Considerations	75
4.4 Frontend Implementation	79
4.4.1 Code Samples	80
Chapter 5: Conclusion and Future Work	84
5.4. Data Engineering Limitations & Future Improvements	85
5.4. AI/DL Limitations & Future Improvements	86
5.4. Backend Limitations & Future Improvements	88
5.4. Frontend Limitations & Future Improvements	89
5.5. Conclusion	90
REFERENCES	92



Where Business Meets Trust Era

Chapter 1

Introduction



1.1 Overview

The rapid growth of social media and digital communication platforms has transformed the way business information is created, shared, and consumed. Every day, millions of users rely on online sources for news related to investments, startups, financial markets, and business opportunities. While this accessibility facilitates knowledge sharing and communication, it also accelerates the spread of fake and misleading business news, which can negatively influence decision-making and reduce trust in digital information.

To address this challenge, this project introduces **Trustra**, an intelligent mobile application that combines Artificial Intelligence (AI), Deep Learning (DL), Natural Language Processing (NLP), and Big Data technologies to detect fake business news and provide entrepreneurs with reliable business support. The application enables users to submit business-related news articles or social media content, which are analyzed using a fine-tuned **DistilBERT** model to determine whether the information is genuine or misleading.

In addition to fake news detection, Trustra integrates **Trusty**, an AI-powered business expert chatbot that assists entrepreneurs and business owners by providing guidance on business planning, feasibility studies, SWOT analysis, marketing strategies, financial management, and startup development. The application also includes a trusted business community feed that shares verified information while limiting user interactions to likes only, reducing the spread of misinformation through comments and discussions.

To support both analytics and model development, the project implements an end-to-end Big Data pipeline that ingests, processes, and transforms data into analytics-ready datasets, enabling Business Intelligence reporting and continuous AI model improvement.

1.2 Executive Summary

Trustra is an AI-powered mobile application designed to combat the spread of fake business news while supporting entrepreneurs with intelligent business assistance. The application is developed using **React Native** for the mobile interface and **Flask** as the backend framework, providing secure communication between users, AI services, and data storage components.

The fake news detection module utilizes a fine-tuned **DistilBERT** deep learning model to classify business-related news and social media posts as either genuine or fake. Users can upload or paste text into the application and receive immediate predictions that help them verify information before making important business decisions.

The system also incorporates **Trusty**, an AI-powered chatbot built using the **xAI Grok API**, acting as a virtual business consultant capable of assisting users with business planning, feasibility studies, SWOT analysis, marketing strategies, financial management, and startup-related inquiries. By leveraging the advanced language understanding capabilities of Grok, the chatbot provides intelligent and context-aware responses to support entrepreneurs and business owners in making informed decisions.

To provide valuable business insights, the project includes a scalable Big Data architecture that collects, processes, and transforms data through multiple stages before storing analytics-ready datasets. These datasets are used to generate interactive **Power BI dashboards**, including market research analysis and business intelligence reports that support data-driven decision-making.



Figure 1: Trustra Dashboard

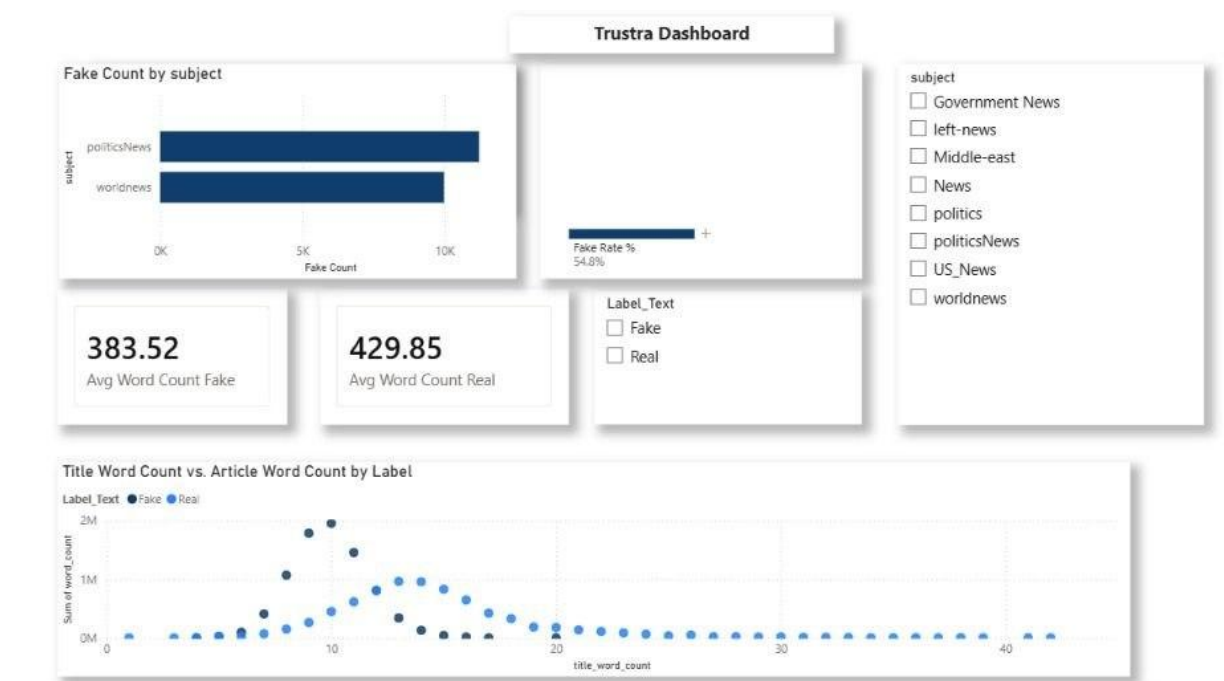


Figure 2: Trustra Dashboard-2

The application's backend is powered by **Flask**, while **PostgreSQL** manages operational data such as user accounts, authentication, community posts, and interactions. By integrating Deep Learning, Big Data, Business Intelligence, and conversational AI into a single platform, Trustra provides a comprehensive solution for improving information credibility and empowering entrepreneurs with trusted business knowledge.



The increasing reliance on digital platforms for business information has made entrepreneurs, investors, and business owners more vulnerable to misinformation. Fake business news, manipulated financial reports, misleading investment advice, and unverified startup information can spread rapidly across social media, leading to poor financial decisions, reputational damage, and economic losses.

Although many users consume online business content daily, verifying the credibility of this information remains challenging due to the large volume of published material and the absence of automated verification mechanisms. In addition, entrepreneurs often require professional business consultation but may face financial or accessibility barriers when seeking expert guidance.

Existing social media platforms also facilitate the rapid dissemination of misinformation through unrestricted user interactions, making it difficult to distinguish between reliable and unreliable content. Consequently, there is a growing need for an intelligent platform capable of automatically detecting fake business news while simultaneously providing accessible business expertise and promoting the distribution of verified information.

1.4 Objectives of the Project

- Develop an intelligent mobile application for detecting fake business news.
- Build and fine-tune a **DistilBERT-based deep learning model** to classify business-related content as genuine or fake.
- Allow users to upload or paste business news and social media text for automated analysis.
- Integrate **Trusty**, an AI-powered business expert chatbot that provides guidance on business planning, feasibility studies, SWOT analysis, marketing strategies, and financial management.
- Design and implement a scalable **Big Data pipeline** to support data ingestion, processing, transformation, and analytics.
- Generate analytics-ready datasets for interactive **Power BI dashboards** and business intelligence reporting.
- Create a trusted business community feed that shares verified information while minimizing misinformation through controlled user interactions.
- Improve users' confidence in online business information and support entrepreneurs in making informed decisions.

1.5 Motivation of the Work

The motivation behind Trustra stems from the growing impact of fake business news on entrepreneurs, investors, and organizations that increasingly depend on digital information for strategic decision-making. The rapid spread of misinformation across social media platforms creates significant challenges for verifying content authenticity and protecting users from inaccurate or manipulated information.

At the same time, many entrepreneurs and startup founders require accessible business guidance but may not have the resources to consult professional advisors. Advances in Artificial Intelligence, Deep Learning, Natural



By combining automated fake news detection, AI-powered business consultation, and scalable data analytics, Trustra aims to enhance information credibility, support entrepreneurial growth, and promote more informed and data-driven business decisions in the digital era.

- Trustra SWOT Analysis



Figure 3: Trustra SWOT Analysis

1.6 Used Tools

1.6.1 Data Engineering Tools

The following table presents the main technologies and tools used in the proposed big data pipeline to help AI-based fake business news detection system.

Layer	Tool / Technology	Purpose
Data Sources	News APIs	Collect business news data from online sources.
Data Sources	Social Media Streams	Gather social media business content and discussions.
Data Sources	Kaggle Datasets	Provide historical datasets for model training and evaluation.
Ingestion	Apache Kafka	Handle real-time data streaming and ingestion.
Data Lake	Amazon S3	Store raw JSON and Parquet files within the data lake.
Processing	Apache Spark	Perform data cleaning, preprocessing, and distributed processing.
Processing	NLP / Feature Engineering	Extract and prepare textual features for analysis.
Data Warehouse	Snowflake	Manage Bronze, Silver, and Gold layers for scalable analytics.
Transformation	dbt	Apply data modeling, transformation, and business logic.
Orchestration	Apache Airflow	Automate and schedule workflow execution across the pipeline.

Table 1: Data Engineering Tools

The proposed architecture integrates big data engineering, cloud storage, workflow orchestration, machine learning, and business intelligence technologies to ensure scalable data processing, reliable analytics, and efficient fake news detection.

1.6.2 AI/DL Tools

The following tools and frameworks were selected to build a production-ready, multi-task fake news detection system. Each choice was motivated by performance, community support, and suitability for NLP research.

Tool / Library	Category	Purpose in Project
PyTorch	Deep Learning Framework	Core framework for building, training, and evaluating the multi-task neural network with full gradient control.
HuggingFace Transformers	NLP Library	Provides the pre-trained DistilBERT model and fast tokenizer for contextual text representations.

DistilBERT	Pre-trained Language Model	Shared encoder backbone (66M parameters, pre-trained on 3B+ tokens) producing rich 768-d contextual embeddings.
scikit-learn	ML Utilities	Label encoding, stratified train/val/test splits, classification reports, confusion matrices, and class weight computation.
Pandas / NumPy	Data Manipulation	Dataset loading, cleaning, exploratory analysis, and array operations throughout the pipeline.
Matplotlib / Seaborn	Visualization	Plotting training curves (loss and accuracy per epoch) and confusion matrices for both task heads.
Flask	API Framework	Serves the trained model via REST endpoints: health check, single prediction, and batch prediction (up to 32 articles).
Uvicorn	ASGI Server	Runs the Flask application in both development (--reload) and production modes.
Pydantic	Data Validation	Validates API request and response schemas, enforcing input constraints such as text length and threshold range.
Google Colab (GPU)	Compute Platform	Cloud-based GPU environment used for training with CUDA mixed-precision (GradScaler / autocast) to accelerate training.

Table 2: AI/DL Tools

1.6.3 Backend Tools

The table below provides a consolidated one-line overview of every tool used across the full backend stack.

Category	Tool	Role
Runtime	Node.js	Server-side JavaScript execution environment
Web Framework	Express.js	HTTP routing and middleware pipeline
Authentication	jsonwebtoken	JWT signing and verification
Security	bcrypt	Password hashing
Security	express-rate-limit	Request throttling (3-tier)
Validation	express-validator	Input validation and error formatting
Database	PostgreSQL	Persistent relational data store
ORM	Sequelize	Model-to-table mapping and query abstraction
AI — Chatbot	xAI Grok API	Large-language-model powering Trusty
AI — News	AI Classifier	Fake-news detection and categorisation
HTTP Client	Axios	Outbound requests to AI APIs
Media	Base64 / Buffer	Avatar image encoding and validation
Cloud Hosting	Railway	Managed deployment with auto-scaling
Config	dotenv	Environment variable management
Dev Tools	Postman / Nodemon / ESLint	API testing, hot-reload, code quality
Version Control	Git / GitHub	Source control and CI/CD trigger
npm	Package manager	All dependencies are declared in package.json and locked via package-lock.json Bundled with Node.js

Table 3: Backend Tools



1.6.4 Frontend Tools

The Trustra mobile application frontend was developed using modern cross-platform mobile technologies to ensure scalability, maintainability, and responsive user experience across devices.

Technology / Tool	Purpose	Details
React Native	Frontend Framework	Used to build cross-platform mobile interfaces.
Expo SDK 54	Development Platform	Simplifies React Native development and deployment.
JavaScript / JSX	Programming Language	Used for frontend logic and UI components.
TypeScript	Type Safety	Used in index.tsx for better maintainability.
AsyncStorage	Local Storage	Stores JWT access and refresh tokens securely.
Native Fetch API	API Communication	Handles communication with backend REST APIs.
Node.js Backend API	Backend Integration	Provides authentication and news verification services.
JWT Authentication	User Authentication	Used for secure login sessions.
DateTimePicker	UI Component	Allows users to select dates in registration forms.
VS Code	Code Editor	Primary development environment used during implementation.

Table 4: Frontend Tools

1.7 Thesis Structure

This thesis is organized into several chapters:

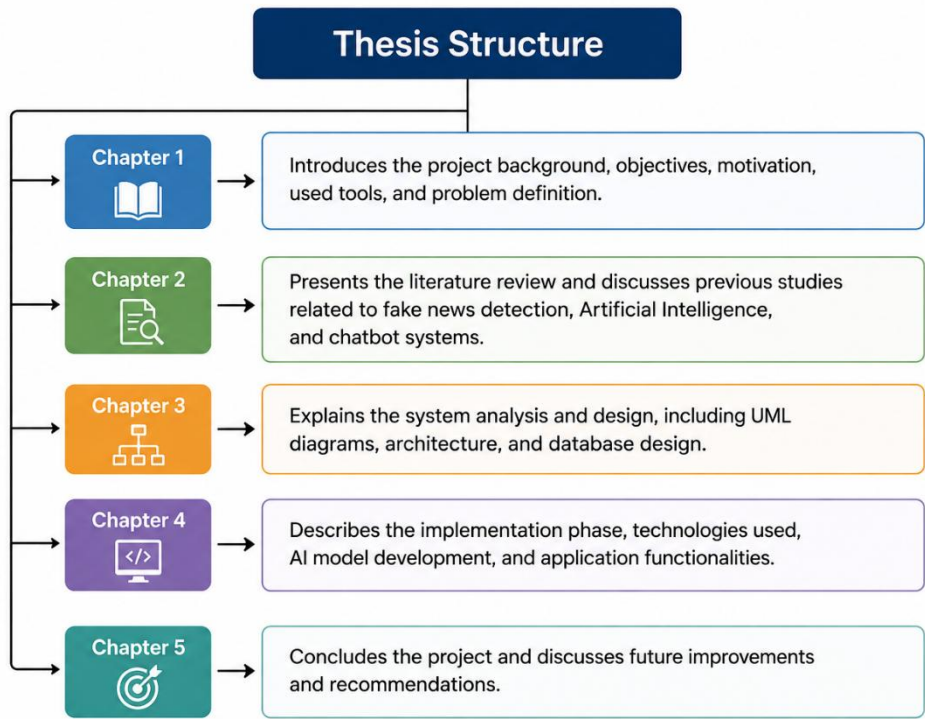


Figure 4: Thesis Structure



Where Business Meets Trust Era

Chapter 2

State of the Art



2.1 Overview

This chapter presents a review of existing systems and platforms relevant to the problem domain addressed by Trustra. The reviewed systems are evaluated with respect to their design objectives, underlying artificial intelligence methodologies, functional capabilities, and limitations. Following the general review, a focused comparative analysis is conducted between Trustra and the two most functionally and technically aligned systems identified in the literature: Logically, a commercial fact-checking platform that combines AI-driven content screening with structured editorial workflows, and FakeNewsNet-Based Systems, a category of academic research systems that employ transformer-based language models for automated fake news classification. This comparative analysis aims to contextualise the design decisions made in the development of Trusty and to demonstrate the extent to which Trustra advances beyond the capabilities of existing solutions.

2.2 Existing Projects

A) Logically:

- System Description

Logically is a commercial fact-checking platform that combines artificial intelligence with the expertise of professional human analysts to identify and refute misinformation across digital media. The platform is deployed primarily at an institutional and governmental level, serving organizations that require scalable, high-accuracy fact-checking capabilities. Logically processes content from a wide variety of sources, including social media platforms, news websites, and messaging applications, and produces structured fact-check reports that contextualise claims within broader evidential frameworks. The platform places particular emphasis on the integration of evidence retrieval systems that surface supporting or contradictory material from verified sources.

- AI Implementation and Methodology

Logically employs a multi-layered AI infrastructure that incorporates natural language processing, knowledge graph integration, and automated evidence retrieval pipelines. The platform's AI components are responsible for the initial identification and triaging of potentially misleading content, after which human fact-checkers conduct detailed investigations and produce authoritative assessments. Knowledge graphs allow Logically to situate identified claims within a structured network of established facts, facilitating more contextually aware analysis. The combination of AI automation and human oversight is central to Logically's methodology, enabling the platform to scale its operations while maintaining editorial accountability.

- Main Advantages

Logically's most notable strengths are its multilingual support, its evidence-based verification methodology, and the authority conferred by its hybrid AI-human approach. The platform is capable of operating across multiple languages and cultural contexts, which significantly expands its applicability in international environments. The combination of automated AI screening and expert human review produces assessments that are both efficient

and credible. Evidence retrieval capabilities allow Logically to substantiate its verdicts with references to verifiable sources, enhancing the trustworthiness and transparency of its outputs. The platform's commercial-scale infrastructure enables it to process high volumes of content within operationally demanding environments.

- Main Limitations

Logically is oriented primarily towards institutional clients and government partners, which means it is not designed as a community-facing publishing platform through which ordinary users can share and verify news content. The platform does not offer a direct news-sharing interface for general users, nor does it incorporate a conversational chatbot for interactive engagement. Its dependence on human fact-checkers introduces latency into the verification process that may be unsuitable for contexts requiring near-instantaneous results. Furthermore, Logically's commercial positioning means that its services are not freely accessible to individual users or small organizations, limiting its reach as a public-facing tool for media literacy.

- Comparison with Trustra

Trustra and Logically share a common commitment to combating misinformation, but they are designed for substantially different deployment contexts and user bases. Logically operates as a commercial service targeting institutional clients, whereas Trustra is designed as a publicly accessible application that enables individual users to share, read, and verify news articles within a moderated community environment. Trustra's fake news detection component, powered by a fine-tuned DistilBERT model, provides automated classification with a confidence score that is immediately available to end users upon article submission. The verification before publication workflow in Trustra ensures that community-submitted content is screened prior to publication, fulfilling a function analogous to the human review stage in Logically's pipeline. The Trusty Chatbot further distinguishes the application's architecture from Logically's commercially oriented service model.

- Advantages of Trustra Over Logically

Trustra's principal advantage over Logically is its accessibility to general users without institutional affiliation or subscription requirements. The community-based news sharing application enables public engagement with verified content in a way that Logically does not support. The DistilBERT-powered classification model provides real-time feedback on submitted articles, reducing the latency associated with human-in-the-loop verification processes. The integrated chatbot offers users an interactive mechanism for querying the system and serves business, a feature entirely absent from Logically's publicly described service offerings. Trustra's lightweight deployment architecture, facilitated by DistilBERT's computational efficiency, also makes the system more amenable to resource-constrained hosting environments.

- Limitations of Trustra Compared to Logically

Trustra does not employ human fact-checkers or integrate an evidence retrieval mechanism, which means that its classifications are generated solely by the underlying machine learning model without recourse to verifiable supporting sources. Logically's multilingual capabilities also represent a dimension in which Trustra's current implementation is more limited, as the DistilBERT model deployed within Trustra is primarily trained on English-language datasets. The absence of a structured knowledge graph means that Trustra cannot situate identified claims within broader networks of established facts in the manner that Logically's architecture enables.

B) NewsGuard

- System Description

NewsGuard is a credibility rating service that evaluates the reliability of online news sources and websites using a team of professionally trained journalists. Rather than analysing individual articles, NewsGuard assigns numerical credibility scores to entire news domains based on a structured set of journalistic criteria. These criteria encompass editorial transparency, correction policies, the distinction between news and opinion content, and responsible sourcing practices. The resulting ratings are made available to users through a browser extension that visually indicates the trustworthiness of a given news outlet before any article is read. NewsGuard operates with a strong emphasis on human editorial judgement, ensuring that each rating is accompanied by a detailed, human-authored explanation that articulates the reasoning behind the assigned score.

- AI Implementation and Methodology

NewsGuard does not rely on automated machine learning models for its core evaluation process. Instead, the platform employs human journalists who apply a consistent, predefined rubric to assess news websites against standardized journalistic standards. While NewsGuard incorporates some degree of digital tooling for data collection and presentation, the absence of a computational classification model distinguishes it fundamentally from AI-driven systems. The credibility scoring process remains inherently manual, which contributes to its high reliability but simultaneously restricts its scalability and the speed at which new or emerging sources can be reviewed.

- Main Advantages

The principal strengths of NewsGuard lie in its high credibility, transparency, and the explanatory depth of its ratings. Because evaluations are conducted by domain experts with journalistic training, the resulting assessments are generally regarded as authoritative and well-reasoned. The system provides users with clear, evidence-based explanations for each rating, promoting media literacy and informed consumption of news content. Furthermore, the reliance on human reviewers minimizes the risk of algorithmic bias that may affect automated classification

systems. For end users, the browser extension integration offers a seamless and non-intrusive experience that surfaces credibility information at the point of consumption.

- Main Limitations

Despite its strengths, NewsGuard presents several significant limitations that restrict its applicability in fast-moving digital news environments. The platform does not support the classification or verification of individual news articles submitted by ordinary users, which limits its utility for community-driven platforms. Because the evaluation process is conducted manually, NewsGuard cannot scale rapidly to accommodate the volume of new content published daily across the internet. The system does not offer an automated confidence score or a real-time classification mechanism, which constrains its integration potential within applications that require immediate feedback. Additionally, NewsGuard's coverage is inherently bounded by the number of websites that its editorial team has reviewed, leaving substantial portions of the online news landscape unassessed.

- Comparison with Trustra

Trustra differs fundamentally from NewsGuard in both scope and methodology. While NewsGuard focuses exclusively on website-level credibility assessment conducted by human professionals, Trustra performs automated, article-level fake news detection and category classification using a fine-tuned DistilBERT transformer model. This enables Trustra to evaluate individual pieces of content in real time, regardless of the publication source. Furthermore, Trustra incorporates a community-based news-sharing environment where registered users can submit and read news articles, subject to a verification-before-publication workflow that ensures content is screened prior to public dissemination. The inclusion of a Trusty Chatbot provides users with an interactive interface for querying news-related information and business needs, a feature entirely absent from NewsGuard's service model. An administrative dashboard further supports content moderation at scale, offering oversight capabilities that complement the automated detection pipeline.

- Advantages of Trustra Over NewsGuard

Trustra's primary advantage over NewsGuard is its capacity for automated, real-time analysis of individual news articles, enabling rapid decision-making without dependency on human reviewers. The fine-tuned DistilBERT model produces a quantitative confidence score that allows users to assess the probability that a given article is misleading or fabricated. The community engagement features, chatbot interface, and verification-before-publication workflow represent functional dimensions that fall entirely outside NewsGuard's operational scope. Trustra is also inherently more scalable, as its deep learning infrastructure can process large volumes of submissions simultaneously.

- Limitations of Trustra Compared to NewsGuard

Trustra does not incorporate human expert review, which means that its classifications, while statistically informed, lack the contextual nuance and editorial accountability that professional journalists can provide.

NewsGuard's detailed, human-authored explanations for each rating offer a level of interpretability and transparency that automated models, including DistilBERT, cannot fully replicate. In domains where explainability is critical, such as legal or regulatory contexts, NewsGuard's approach retains a comparative advantage.

2.3 Comparative Analysis Summary

The three systems examined in this chapter are Logically, NewsGuard, and Trustra. Each represent a distinct paradigm within the fake news mitigation landscape. Logically demonstrates how AI automation and human editorial review can be combined to produce a credible, scalable fact-checking service; however, its institutional focus and commercial model preclude meaningful public community engagement. NewsGuard, by contrast, prioritises editorial transparency and human expertise through journalist-driven credibility ratings, yet its manual evaluation process restricts scalability and renders it incapable of performing real-time, article-level classification for user-submitted content.

Trustra is designed to occupy the space between these two paradigms while extending beyond the limitations of both. It adopts the automated AI-driven classification approach that platforms such as Logically employ at a technical level, while simultaneously offering the public accessibility and community engagement that NewsGuard's expert-review model does not support. Trustra achieves this through a fully operational, publicly accessible application that integrates community news sharing, a verification-before-publication workflow, conversational AI interaction via the Trusty Chatbot. Nevertheless, Trustra is not without limitations. Unlike Logically, it does not incorporate human fact-checkers or an evidence retrieval mechanism, meaning that classification decisions rest entirely on the underlying machine learning model without recourse to verifiable external sources. Unlike NewsGuard, it does not provide human-authored explanatory narratives that contextualise why a piece of content has been flagged, which reduces the interpretability of its outputs for non-technical users.

The selection of DistilBERT as Trustra's core classification model reflects a principled balance among four criteria: classification accuracy, computational efficiency, model size, and real-time inference performance. DistilBERT retains approximately ninety-seven percent of BERT-base's language understanding capability while reducing model size by forty percent and improving inference speed by sixty percent. This profile makes it substantially more suitable than full-scale transformer models for deployment within an application that must deliver immediate classification feedback to users. However, this efficiency advantage comes at a measurable cost; DistilBERT does not achieve the classification accuracy of larger models such as RoBERTa or XLNet, and the system currently operates primarily on English-language content, limiting its applicability in multilingual contexts where platforms such as Logically maintain a comparative advantage. By pr21enningsen21seployability alongside strong classification performance, Trustra achieves a practical utility that neither Logically's institutionally gated service nor NewsGuard's manually curated rating system fully provides, while acknowledging that further development is required to close the gap in explainability and language coverage.



Where Business Meets Trust Era

Chapter 3

System Analysis and Design



Chapter 3: System Analysis and Design

3.1 Overview

This chapter presents the system analysis and design of Trustra, a mobile application developed for the detection of fake business news. The chapter is structured to cover the full spectrum of system definition — from requirements elicitation through to architectural and behavioral design — providing a comprehensive blueprint of how the application is conceived, structured, and intended to operate.

The system analysis section (3.2) begins by establishing the project requirements, capturing both functional and non-functional specifications. Functional requirements are divided into user-facing capabilities — such as community browsing, post management, news credibility checking, and chatbot interaction — and system-level responsibilities, including authentication, content moderation, the AI detection engine, and data backup. Non-functional requirements define the quality standards the system must uphold across six dimensions: performance, security, reliability and availability, usability, maintainability, and compatibility.

The system design section that follows translates these requirements into concrete architectural and behavioral models. A set of UML diagrams is used to capture the structure and dynamics of the system from multiple perspectives. Use case diagrams illustrate the interactions between actors and the system's core functionalities. Class diagrams define the data model and the relationships between system entities. Sequence diagrams trace the flow of operations across system components for key user scenarios, such as submitting an article for credibility analysis or interacting with the Trusty chatbot. Activity diagrams model the logical flow of processes such as user registration and news verification. The deployment diagram then maps the system's physical architecture, showing how application components, servers, databases, and third-party services are distributed and connected in the production environment.

Together, the analyses and diagrams in this chapter serve as the technical foundation for the implementation phase, ensuring that every feature described in the requirements is accounted for in the design and that the system is built on a coherent, scalable, and well-documented architecture.

3.2 System Analysis

3.2.1 Project Requirements

A) Functional Requirements

1. Access community feed

User: The user and admin can view the community posts.

System: The system shall retrieve and display all available community posts to the user.

2. Sign Up

User: The user and admin can sign up for a new account.

System: The system shall validate the user's input, create a new account, and store the user's credentials securely.



User: The user and admin can log in to their account.

System: The system shall check the entered credentials against stored records and grant access if they match, or display an error message if they do not.

4. Update Personal Profile

User: The user and admin can update their personal profile information.

System: The system shall allow the user to modify their profile details and save the changes to the database.

5. Request Detection

User: The user and admin can submit a news article or content for fake news detection.

System: The system shall require the user to be logged in, then forward the submitted content to the AI model and return the detection result to the user.

6. Post News

User: The user and admin can post news to the community.

System: The system shall validate the news content and publish it to the community feed, making it visible to other users.

7. Share News

User: The user and admin can share an existing news post.

System: The system shall create a shared copy of the selected post and display it on the user's profile and community feed.

8. Like Shared Post

User: The user and admin can like a shared post.

System: The system shall record the user's like and update the like count on the shared post accordingly.

9. Save Posts

User: The user and admin can save posts for later viewing.

System: The system shall store the selected post in the user's saved posts list and allow the user to access it at any time.

10. Search News



User: The user and admin can search for news using keywords.

System: The system shall query the database for matching news articles and display the relevant results to the user.

11. Search Users

User: The user and admin can search for other users by name or username.

System: The system shall query the database for matching user profiles and display the results to the user.

12. Use Trusty Chatbot

User: The user and admin can interact with the Trusty Chatbot by sending messages.

System: The system shall forward the user's message to the chatbot, return the chatbot's response, and automatically save the conversation in the user's chat history.

13. Delete Account

User: The user and admin can request to delete their account.

System: The system shall ask for confirmation, then permanently remove the user's account and all associated data from the database.

14. Logout

User: The user and admin can log out of their account.

System: The system shall terminate the user's active session and redirect them to the login or home page.

12. Monitor System Dashboard

User: The admin can monitor the system dashboard.

System: The system shall retrieve and display real-time system statistics, activity logs, and performance metrics on the admin dashboard.

13. Manage Users

User: The admin can manage all registered users in the system.

System: The system shall provide the admin with user management tools including the following:

14. Filter Users

User: The admin can filter users based on specific criteria.

System: The system shall apply the selected filters and display the matching users accordingly.

15. Get User Information

User: The admin can view detailed information about a specific user.

System: The system shall retrieve and display the full profile and activity details of the selected user.

16. Edit User

User: The admin can edit a user's information.

System: The system shall allow the admin to modify the selected user's details and save the changes to the database.

17. Delete User

User: The admin can delete a user's account.

System: The system shall ask for confirmation, then permanently remove the selected user's account and all associated data from the database.

18. Train dataset

User: The admin can export the dataset and train the AI model using an external training environment.

System: The system shall integrate and deploy the trained model produced by the admin into the application.

18. Manage Posts

User: The admin can manage all posts published in the system.

System: The system shall provide the admin with post management tools including the following:

19. Filter Posts

User: The admin can filter posts based on specific criteria.

System: The system shall apply the selected filters and display the matching posts accordingly.

20. Get Post Information

User: The admin can view detailed information about a specific post.

System: The system shall retrieve and display the full details and engagement metrics of the selected post.

21. Edit Post

User: The admin can edit a post's content.

System: The system shall allow the admin to modify the selected post's details and save the changes to the database.

22. Delete Post

User: The admin can delete a post.

System: The system shall ask for confirmation, then permanently remove the selected post and all associated data from the database.

23. Detect News

User: The user and admin can write news content to detect whether it is real or fake news.

System: The system shall receive the submitted content, pass it to the AI model for analysis, and return the detection result along with a confidence score to the requesting user or admin.

24. Classify News

Use: The user and admin can write news content to detect its category.

System: The system shall pass the news content to the AI model, receive the predicted category, and display the classification result to the requesting user or admin.

25. Reply to Messages

User: The user and admin can write messages to ask about anything especially business content.

System: The system shall receive the user's message, forward it to the chatbot engine, retrieve the generated response, and display it back to the user in real time.

26. Retrieve Previous Conversations

User: The Trusty Chatbot can retrieve and display previous conversations for the user.

System: The system shall query the chat history database for all past conversations associated with the user and display them in chronological order within the chatbot interface.

B) NON-FUNCTIONAL REQUIREMENTS

The following non-functional requirements define the quality attributes and operational characteristics of the proposed system:

1. Performance

- The system shall analyze and classify submitted business news within an acceptable response time under normal operating conditions.



- The application shall efficiently process user requests while maintaining stable performance.
2. **Scalability**
 - The system shall support an increasing number of users and growing volumes of business news data without significant degradation in performance.
 - The Big Data pipeline shall be capable of handling large datasets for analytics and model training.
 3. **Security**
 - User authentication and authorization shall be implemented to ensure secure access to the application.
 - Sensitive user information and credentials shall be securely stored and protected from unauthorized access.
 - Communication between the mobile application and backend services shall be encrypted using secure protocols.
 4. **Availability**
 - The application shall be available to users whenever internet connectivity and backend services are operational.
 - The backend services should minimize downtime and ensure reliable access to system functionalities.
 5. **Reliability**
 - The system shall consistently provide accurate and dependable operation without unexpected failures.
 - Data processing and storage operations shall maintain integrity throughout the application's lifecycle.
 6. **Usability**
 - The mobile application shall provide an intuitive and user-friendly interface suitable for users with different technical backgrounds.
 - Navigation between application features shall be simple and easy to understand.
 7. **Maintainability**
 - The system architecture shall be modular to facilitate future updates, debugging, and feature enhancements.
 - Source code shall be well organized and documented to simplify maintenance.
 8. **Compatibility**
 - The mobile application shall be compatible with modern Android devices and support different screen sizes.
 - The backend APIs shall support communication with the React Native application through standard RESTful interfaces.
 9. **Data Integrity**
 - The system shall ensure that stored data remains accurate and consistent during processing and database transactions.
 - Data transformations within the Big Data pipeline shall preserve data quality before analytics and model training.
 10. **Extensibility**
 - The architecture shall support the integration of additional AI models, business services, or analytics components in future releases without major modifications to the existing system.

- Overview

Use Case:

A use case represents a specific interaction between a user and the system to achieve a particular goal. It describes what the user wants to do and how the system responds.

Use Case Description:

A use case description provides detailed information about the use case, including the actor involved, the steps performed, the expected outcome, and any conditions or requirements.

A) Use Cases:

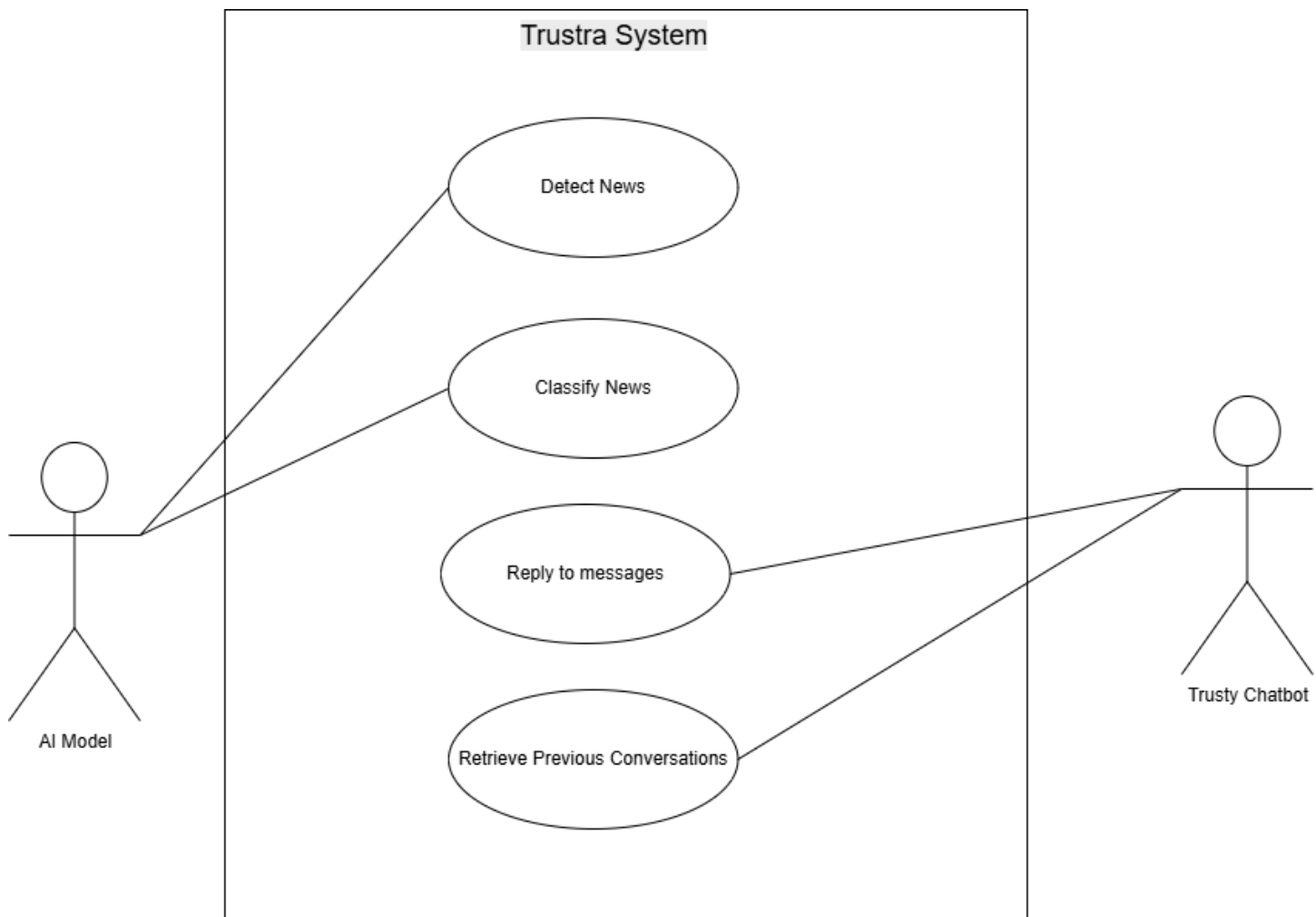


Figure 5: Use case-1

Trustra System

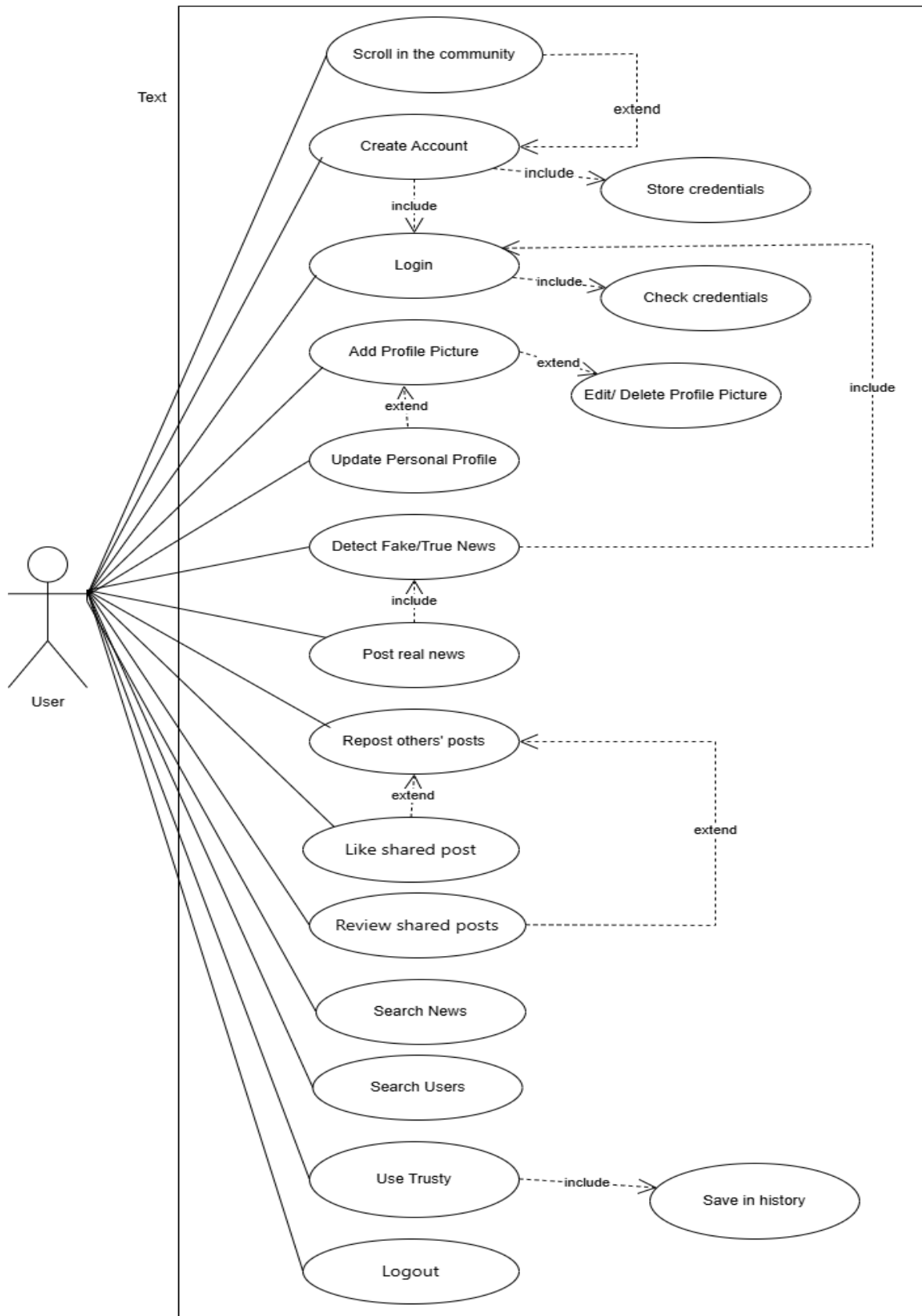


Figure 6: Use case-2



Figure 7: Use case-3

B) Use Case Descriptions:

Use case name:	Access community feed	
Scenario:	Viewing posts shared in the community feed	
Triggering event:	The user wants to browse community content	
Brief description:	The user can view posts shared by other community members in the platform feed	
Actors:	User	
Related use cases:	Search News, Like shared post, Save posts	
Stakeholders:	Users	
Post-conditions:	User can interact with posts (like, save, share)	
Flow of activities:	Actor	System
	1. User navigates to the community feed	1. System retrieves and displays the latest community posts
Exception conditions:	1. No posts available – system displays an empty state message. 2. Network error – system notifies the user.	

Use case name:	Signup	
Scenario:	Creating a new account on the platform	
Triggering event:	A new user wants to register on the platform	
Brief description:	A new user fills in his/her personal information to create an account; the system stores credentials securely	
Actors:	User	
Related use cases:	Store credentials, Login	
Stakeholders:	Users	
Pre-conditions:	User does not already have an account User has a valid email address	
Post-conditions:	Account is created User can log in with the new credentials	
Flow of activities:	Actor	System

	1. User opens the signup page 2. User fills in name, email, and password 3. User submits the form	1. System displays the registration form 2. System validates input fields 3. System stores credentials and sends a confirmation
Exception conditions:	1. Email already registered – system prompts the user to log in. 2. Invalid email format – system shows a validation error.	

Use case name:	Login	
Scenario:	Authenticating an existing user	
Triggering event:	The user wants to access their account	
Brief description:	The user provides credentials and the system checks them to grant access to the application	
Actors:	User, Admin	
Related use cases:	Check credentials	
Stakeholders:	Users, Admins	
Pre-conditions:	User must have a registered account	
Flow of activities:	Actor	System
	1. User enters email and password 2. User clicks the login button	1. System validates input format 2. System checks credentials against the database and accept or reject access
Exception conditions:	1. Incorrect password – system displays an error message. 2. Account not found – system suggests signup.	

Use case name:	Update Personal Profile
Scenario:	Editing user profile information
Triggering event:	The user wants to update their personal information
Brief description:	The user can modify their profile details such as name, photo, and bio

Actors:	User, Admin	
Related use cases:	Login	
Stakeholders:	Users, Admin	
Pre-conditions:	User must be logged in	
Flow of activities:	Actor	System
	1. User navigates to profile settings 2. User edits desired fields 3. User saves changes	1. System loads current profile data 2. System validates and saves updated profile information
Exception conditions:	1. Invalid input format – system shows validation errors.	

Use case name:	Request Detection	
Scenario:	Requesting AI-based news detection	
Triggering event:	The user or admin triggers a news detection request	
Brief description:	The user or admin requests the AI model to detect real or fake news from available sources	
Actors:	User	
Related use cases:	Post news	
Stakeholders:	Users, AI Model	
Pre-conditions:	User must be logged in	
Flow of activities:	Actor	System
	1. User submits a detection request	1.1 System forwards the request to the AI Model 1.2 System receives and displays detection results
Exception conditions:	1. AI Model unavailable – system notifies the user.	

Use case name:	Post news
Scenario:	Publishing a news article on the application after checking that it is real

Triggering event:	The user wants to share real news after detection	
Brief description:	The user verify news using our AI model and publishes the real one to becomes visible to the community	
Actors:	User	
Related use cases:	Request detection, Share news	
Stakeholders:	Users	
Pre-conditions:	User must be logged in	
Flow of activities:	Actor	System
	1. User clicks 'P'st News' 2. User fills in title and content 3. User submits the post	1. System validates and publishes the post

Use case name:	Share News	
Scenario:	Sharing an existing news post	
Triggering event:	The user wants to reshare a post with others	
Brief description:	The user shares a news post, which extends its reach within the community	
Actors:	User	
Related use cases:	Post verified news, Like shared post	
Stakeholders:	Users	
Pre-conditions:	User must be logged in A news post must exist	
Post-conditions:	Post is reshared and appears in the user's' feed	
Flow of activities:	Actor	System
	1. User clicks the share button on a news post	1. System reposts the news to the user's' followers and updates share count
Exception conditions:	1. Post deleted before sharing – system shows a not-found error. 2. Sharing restricted by post owner – system blocks the action.	

Use case name:	Like Shared Post	
Scenario:	Liking a post in the community feed	
Triggering event:	The user wants to like a post	
Brief description:	The user likes a shared post, and the system records and reflects the updated like count	
Actors:	User	
Related use cases:	Share news, Save posts	
Stakeholders:	Users	
Pre-conditions:	User must be logged in Post must exist	
Post-conditions:	Like is recorded and the like count is updated	
Flow of activities:	Actor	System
	1. User clicks the like button on a post	1. System records the like and updates the count
Exception conditions:	1. Already liked – system toggles to unlike. 2. Post no longer available – system shows an error.	

Use case name:	Save Posts	
Scenario:	Saving a post for later viewing	
Triggering event:	The user wants to save a post	
Brief description:	The user saves a post to their saved collection for later reference	
Actors:	User	
Related use cases:	Like shared post, Access community feed	
Stakeholders:	Users	
Pre-conditions:	User must be logged in Post must exist	
Flow of activities:	Actor	System
	1. User clicks the save button on a post	1. System adds the post to the user's saved list
Exception conditions:	1. Post already saved – system toggles unsaved.	

	2. Post deleted – system notifies the user.
--	---

Use case name:	Search News	
Scenario:	Searching for news articles by keyword	
Triggering event:	The user wants to find specific news content	
Brief description:	The user enters a search query and the system returns matching news articles	
Actors:	User	
Stakeholders:	Users	
Pre-conditions:	User must be logged in	
Flow of activities:	Actor	System
	1. User enters a search keyword 2. User submits the search	1. System queries the database and returns matching articles
Exception conditions:	1. No results found – system shows a no-results message.	

Use case name:	Search Users	
Scenario:	Searching for other users on the application	
Triggering event:	The user wants to find another user's profile	
Brief description:	The user enters a name or username to search for other users on the application	
Actors:	User	
Stakeholders:	Users	
Pre-conditions:	User must be logged in	
Flow of activities:	Actor	System
	1. User enters a name or username in the search bar	1. System searches for matching users and returns results
Exception conditions:	1. No users found – system shows an empty result.	

Use case name:	Use Trusty Chatbot	
Scenario:	Interacting with the AI-powered chatbot	
Triggering event:	The user wants to ask business questions or get assistance	
Brief description:	The user interacts with Trusty Chatbot to ask business-related questions; conversations saved to history	
Actors:	User	
Related use cases:	Save in history	
Stakeholders:	Users, Trusty (Chatbot)	
Pre-conditions:	User must be logged in	
Post-conditions:	User receives a response from the chatbot Conversation saved to history	
Flow of activities:	Actor	System
	1. User press on the chatbot icon (button) 2. User types a message and sends it	1. System loads the chat interface 2. System forwards message to the AI model and displays the response
Exception conditions:	1. Chatbot unavailable – system shows an error message.	

Use case name:	Delete Account	
Scenario:	Permanently deleting a user account	
Triggering event:	The user wants to remove their account from the application	
Brief description:	The user initiates account deletion; the system removes all associated data after confirmation	
Actors:	User, Admin	
Stakeholders:	Users, Admins	
Pre-conditions:	User must have an account	
Post-conditions:	Account and all associated data are permanently deleted User is logged out	
Flow of activities:	Actor	System

	1. User navigates to account settings 2. User selects 'Delete Account' 3. User confirms deletion	1. System displays a confirmation prompt 2. System deletes the account and all data from the database, then logs the user out
Exception conditions:	1. User cancels confirmation – account is not deleted. 2. Deletion fails – system notifies the user and retains the account.	

Use case name:	Logout	
Scenario:	Ending the user session	
Triggering event:	The user wants to sign out of the platform	
Brief description:	The user logs out and the system terminates the active session	
Actors:	User, Admin	
Related use cases:	Login	
Stakeholders:	Users, Admins	
Pre-conditions:	User must be logged in	
Post-conditions:	Session is terminated User is redirected to the login page	
Flow of activities:	Actor	System
	1. User clicks the logout button	1. System invalidates the session token and redirects the user to the login page

Use case name:	Monitor System Dashboard	
Scenario:	Viewing the application analytics and system statistics	
Triggering event:	Admin wants to review application performance and activity	
Brief description:	The admin accesses a dashboard displaying system metrics such as active users, post counts, and detection reports	
Actors:	Admin	
Related use cases:	Manage users, Manage posts	

Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in	
Flow of activities:	Actor	System
	1. Admin navigates to the system dashboard	1. System aggregates and displays platform metrics including user activity, post counts, and alerts
Exception conditions:	1. Data retrieval error – system shows a partial dashboard with an error notice. 2. No data available – system shows an empty dashboard.	

Use case name:	Manage Users	
Scenario:	Administering user accounts on the platform	
Triggering event:	Admin wants to manage application users	
Brief description:	The admin can filter, view, edit, or delete user accounts	
Actors:	Admin	
Related use cases:	Filter users, Get user information, Edit user, Delete users	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in	
Post-conditions:	User accounts are updated according to admin actions	
Flow of activities:	Actor	System
	1. Admin opens user management panel	1. System loads the list of registered users with management options
Exception conditions:	1. No users found – system displays an empty list.	

Use case name:	Filter Users	
Scenario:	Filtering user accounts by specific criteria	
Triggering event:	Admin wants to narrow down user list by criteria such as status or role	

Brief description:	The admin applies filters to the user list to find specific accounts	
Actors:	Admin	
Related use cases:	Manage users	
Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in	
Flow of activities:	Actor	System
	1. Admin selects filter criteria (e.g., role, status)	1. System applies the filters and returns the matching user list
Exception conditions:	1. No users match the filter – system shows an empty result. 2. Invalid filter input – system shows a validation error.	

Use case name:	Get User Information	
Scenario:	Viewing detailed information about a specific user	
Triggering event:	Admin wants to inspect a user's profile and activity	
Brief description:	The admin selects a user to view their full profile details and activity history	
Actors:	Admin	
Related use cases:	Manage users, Edit user	
Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in User account must exist	
Flow of activities:	Actor	System
	1. Admin selects a user from the list	1. System retrieves and displays full profile and activity data for the selected user
Exception conditions:	1. User account not found – system shows a not-found error. 2. Data retrieval failure – system shows an error.	

Use case name:	Edit User
Scenario:	Modifying a user's account details

Triggering event:	Admin wants to update a user's information or change their role/status	
Brief description:	The admin edits a user's account data such as role, status, or profile fields	
Actors:	Admin	
Related use cases:	Get user information, Manage users	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in User account must exist	
Flow of activities:	Actor	System
	1. Admin opens a user's detail page 2. Admin modifies fields and saves	1. System loads editable user fields 2. System validates and saves the updated user data
Exception conditions:	1. Invalid input – system shows validation errors. 2. Save failure – system notifies the admin.	

Use case name:	Delete Users	
Scenario:	Removing a user account from the platform	
Triggering event:	Admin wants to permanently delete a user account	
Brief description:	The admin deletes a user account and all associated data from the application	
Actors:	Admin	
Related use cases:	Manage users	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in Target user account must exist	
Post-conditions:	User account and all associated data are permanently deleted	
Flow of activities:	Actor	System
	1. Admin selects a user and clicks 'Delete' 2. Admin confirms deletion	1. System shows a confirmation dialog 2. System permanently deletes the user and their data

Exception conditions:	1. Admin cancels – account is not deleted. 2. Deletion failure – system notifies the admin.
-----------------------	--

Use case name:	Train Dataset	
Scenario:	Admin trains the AI model on the collected dataset using an external environment	
Triggering event:	Admin wants to train the AI model on the prepared dataset.	
Brief description:	The admin trains the AI model on the application's dataset using an external training environment such as Google Colab, producing a trained model ready for deployment.	
Actors:	Admin	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in; A valid dataset must be available or uploaded	
Post-conditions:	A trained AI model is produced and ready to be integrated into the application	
Flow of activities:	Actor	System
	1. Admin exports or accesses the dataset 2. Admin runs the training process on the external environment 3. Admin obtains the trained model	1. Dataset is made available for the external environment 2. External environment trains the model on the dataset 3. Trained model is saved and deployed then integrate to the application
Exception conditions:	1. Dataset is incomplete or corrupted admin must fix the dataset before training 2. Training fails in the external environment admin reviews logs and reruns the process	

Use case name:	Manage Posts
Scenario:	Administering news posts on the application
Triggering event:	Admin wants to oversee and manage application posts

Brief description:	The admin can filter, view, edit, or delete news posts on the application	
Actors:	Admin	
Related use cases:	Filter posts, Get post information, Edit post, Delete post	
Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in	
Post-conditions:	Posts are updated or removed according to admin actions	
Flow of activities:	Actor	System
	1. Admin opens post management panel	1. System loads all posts with management options
Exception conditions:	1. No posts found – system shows an empty list.	

Use case name:	Filter Posts	
Scenario:	Filtering posts by specific criteria	
Triggering event:	Admin wants to narrow down posts by status, category, or date	
Brief description:	The admin applies filters to find specific posts	
Actors:	Admin	
Related use cases:	Manage posts, Get post information	
Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in	
Flow of activities:	Actor	System
	1. Admin selects filter criteria (e.g., date, category)	1. System applies filters and returns matching posts
Exception conditions:	1. No posts match filter – system shows empty result. 2. Invalid filter – system shows an error.	

Use case name:	Get Post Information	
Scenario:	Viewing detailed information about a specific post	
Triggering event:	Admin wants to inspect the full details of a post	

Brief description:	The admin selects a post to view its full content and author	
Actors:	Admin	
Related use cases:	Manage posts, Edit post	
Stakeholders:	Admins	
Pre-conditions:	Admin must be logged in Post must exist	
Flow of activities:	Actor	System
	1. Admin selects a post from the list	1. System retrieves and displays full post details
Exception conditions:	1. Post not found – system shows a not-found error.	

Use case name:	Edit Post	
Scenario:	Modifying post content or status	
Triggering event:	Admin wants to update or correct a post	
Brief description:	The admin edits a post's content due to ensure it's ethical or modifies its publication status	
Actors:	Admin	
Related use cases:	Manage posts	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in Post must exist	
Flow of activities:	Actor	System
	1. Admin opens a post's detail page 2. Admin modifies the content and saves	1. System loads editable post fields 2. System validates and saves the updated post

Use case name:	Delete Post	
Scenario:	Removing a post from the application	
Triggering event:	Admin wants to permanently remove a post	

Brief description:	The admin deletes a news post and all its associated data	
Actors:	Admin	
Related use cases:	Manage posts,	
Stakeholders:	Admins, Users	
Pre-conditions:	Admin must be logged in Post must exist	
Post-conditions:	Post is permanently removed from the platform	
Flow of activities:	Actor	System
	1. Admin selects a post and clicks 'Delete' 'Admin confirms deletion	1. System shows a confirmation dialog 2. System permanently deletes the post
Exception conditions:	1. Admin cancels – post is not deleted. 2. Deletion failure – system notifies the admin.	

Use case name:	Detect News	
Scenario:	Categorizing detected news as real or fake	
Triggering event:	User or admin requests news detection	
Brief description:	The AI Model analyzes submitted content to detect whether it is real or fake news	
Actors:	AI Model	
Related use cases:	Classify News	
Stakeholders:	Admins, Users, AI Model	
Post-conditions:	News detection result is returned to the system	
Flow of activities:	Actor	System
	1. System sends content to the AI Model for detection	1. AI Model processes the content and returns a detection result to the user
Exception conditions:	1. AI Model unavailable – system logs an error. 2. Content format unsupported – model returns an error.	

Use case name:	Classify News	
Scenario:	Detect the category of the news (politics, economics, business)	
Triggering event:	News detection result is received	
Brief description:	The AI Model classifies detected news with it's category and returns the classification	
Actors:	AI Model	
Stakeholders:	Admins, Users, AI Model	
Post-conditions:	News is classified and the result is stored and displayed	
Flow of activities:	Actor	System
	1. AI Model receives the detected news content	1. AI Model classifies the news and returns the right category to the application
Exception conditions:	1. AI Model error – classification is not returned.	

Use case name:	Reply to Messages	
Scenario:	Generating chatbot responses to user messages	
Triggering event:	User sends a message to Trusty Chatbot	
Brief description:	Trusty Chatbot processes the user's' message and generates an appropriate reply	
Actors:	Trusty Chatbot	
Stakeholders:	Users, Trusty Chatbot, Admins	
Pre-conditions:	User has sent a message	
Post-conditions:	A reply is generated and sent back to the user	
Flow of activities:	Actor	System
	1. Trusty Chatbot receives the user's' message	1.1 Trusty processes the message 1.2 Trusty sends the generated reply to the user
Exception conditions:	1. Message processing timeout – system notifies the user.	

Use case name:	Retrieve Previous Conversations	
Scenario:	Loading previous chatbot conversation history	
Triggering event:	User opens the chatbot and wants to view past conversations	
Brief description:	Trusty Chatbot retrieves previously saved conversation history so the user can continue or review past interactions	
Actors:	Trusty Chatbot	
Related use cases:	Save in history	
Stakeholders:	Users, Trusty Chatbot, Admins	
Pre-conditions:	User must be logged in Conversation history must exist	
Flow of activities:	Actor	System
	1. User opens the chatbot history panel	1. Trusty Chatbot retrieves saved conversations from the database and displays them
Exception conditions:	1. No previous conversations – system shows an empty state. 2. Data retrieval failure – system shows an error.	

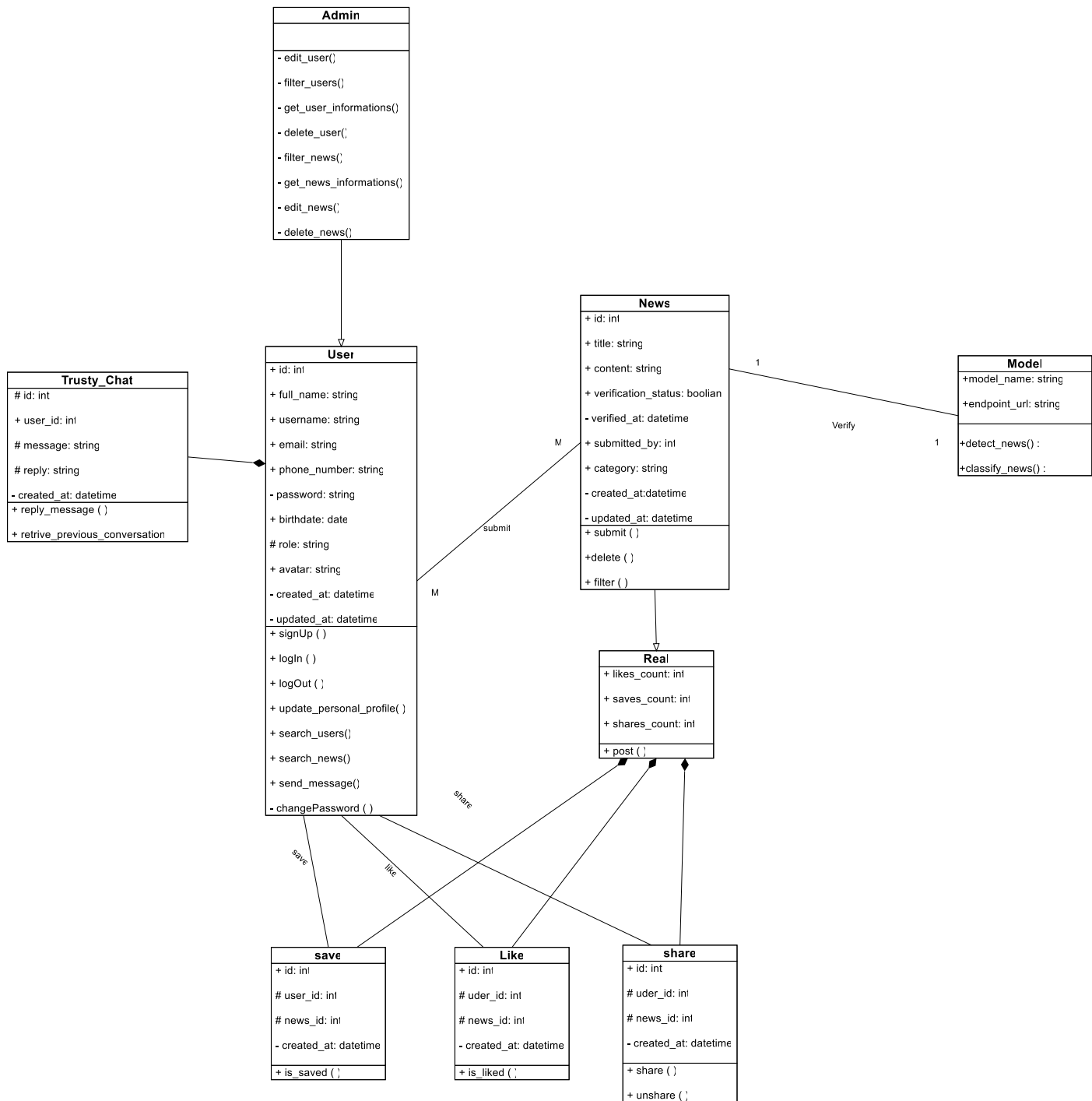


Figure 8: Class Diagram

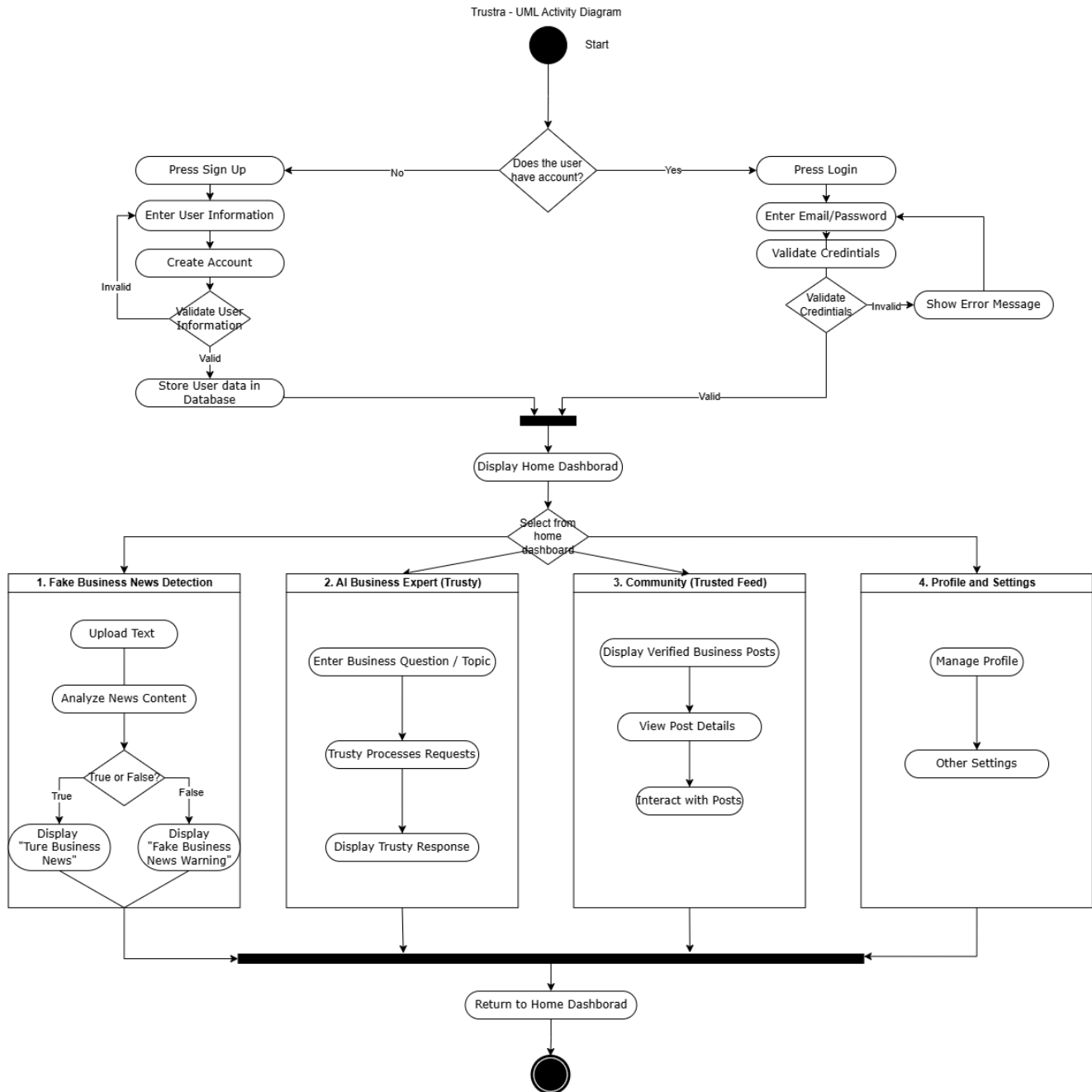


Figure 9: Activity Diagram

3.2.5 Sequence Diagram

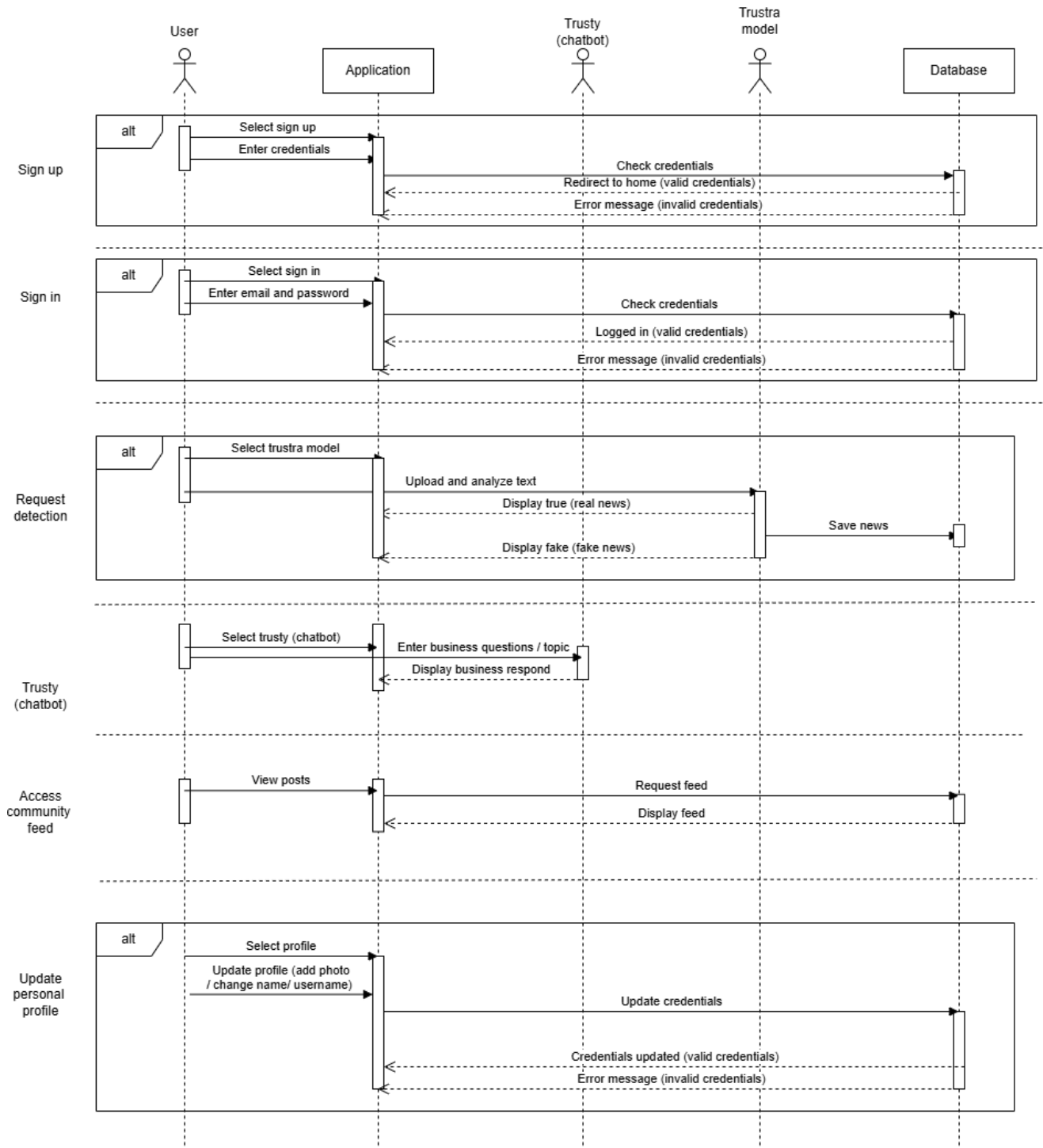


Figure 10: Sequence Diagram

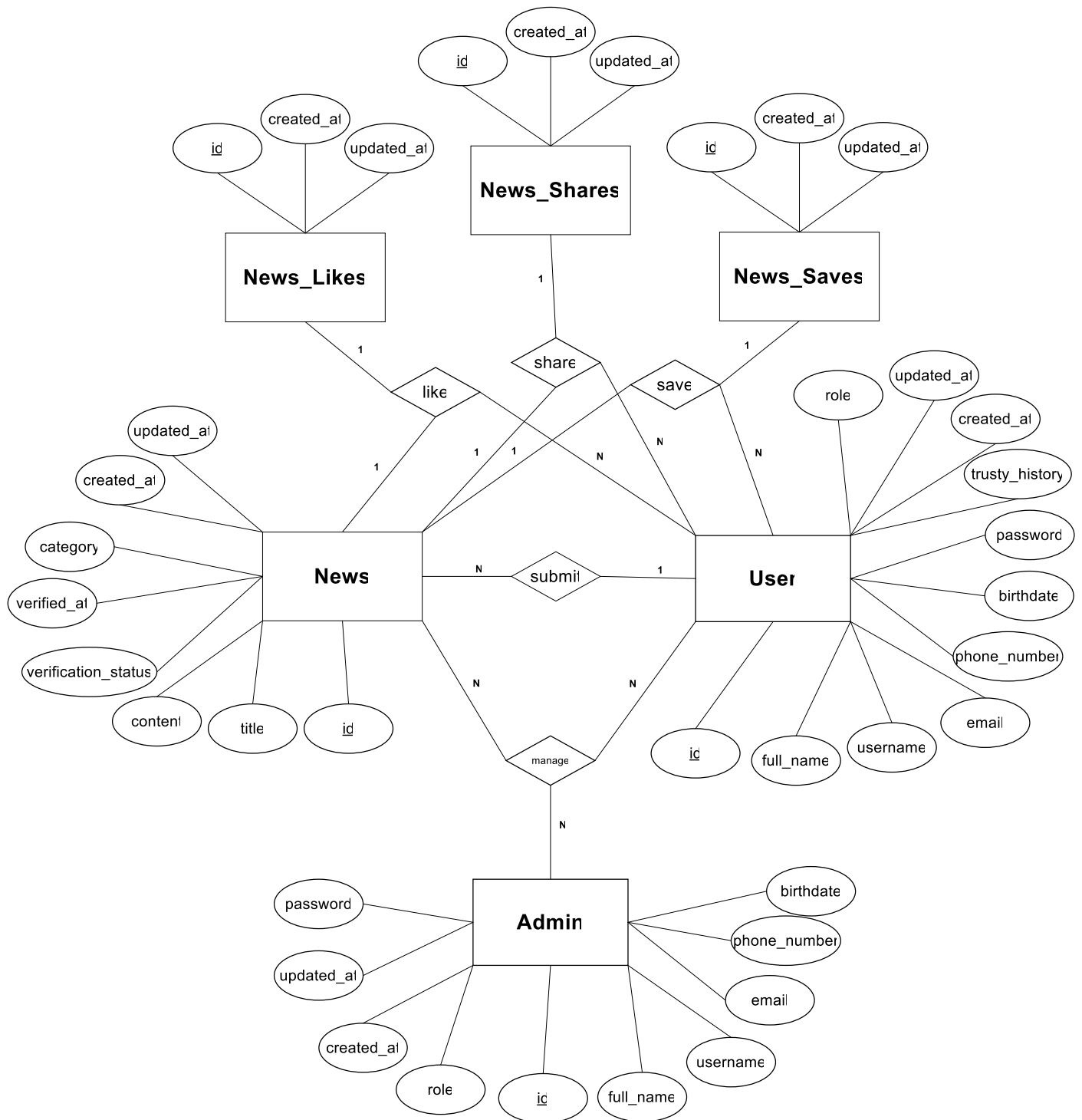


Figure 11: Entity Relationship Diagram

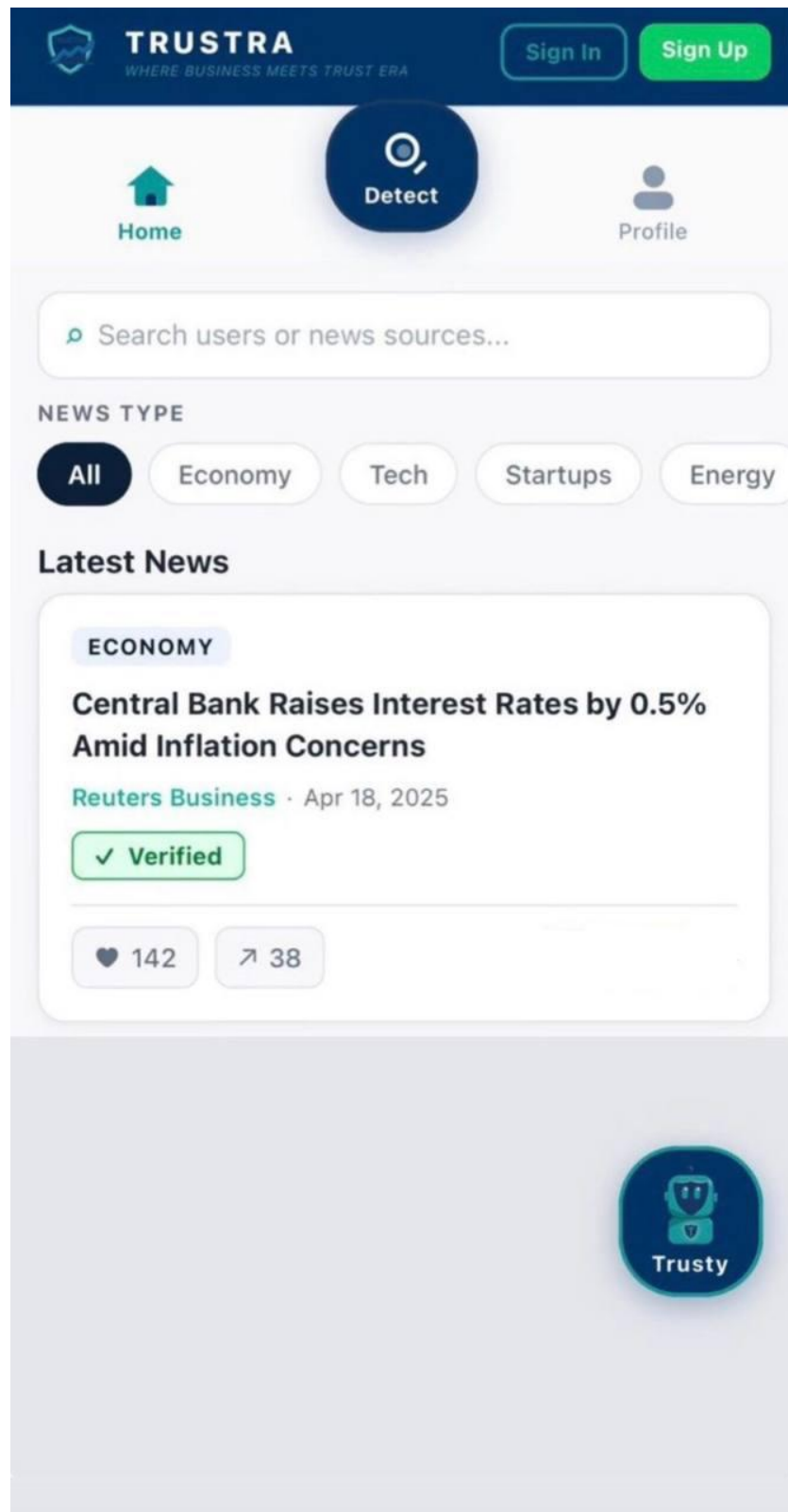


Figure 12: Home screen/Community feed

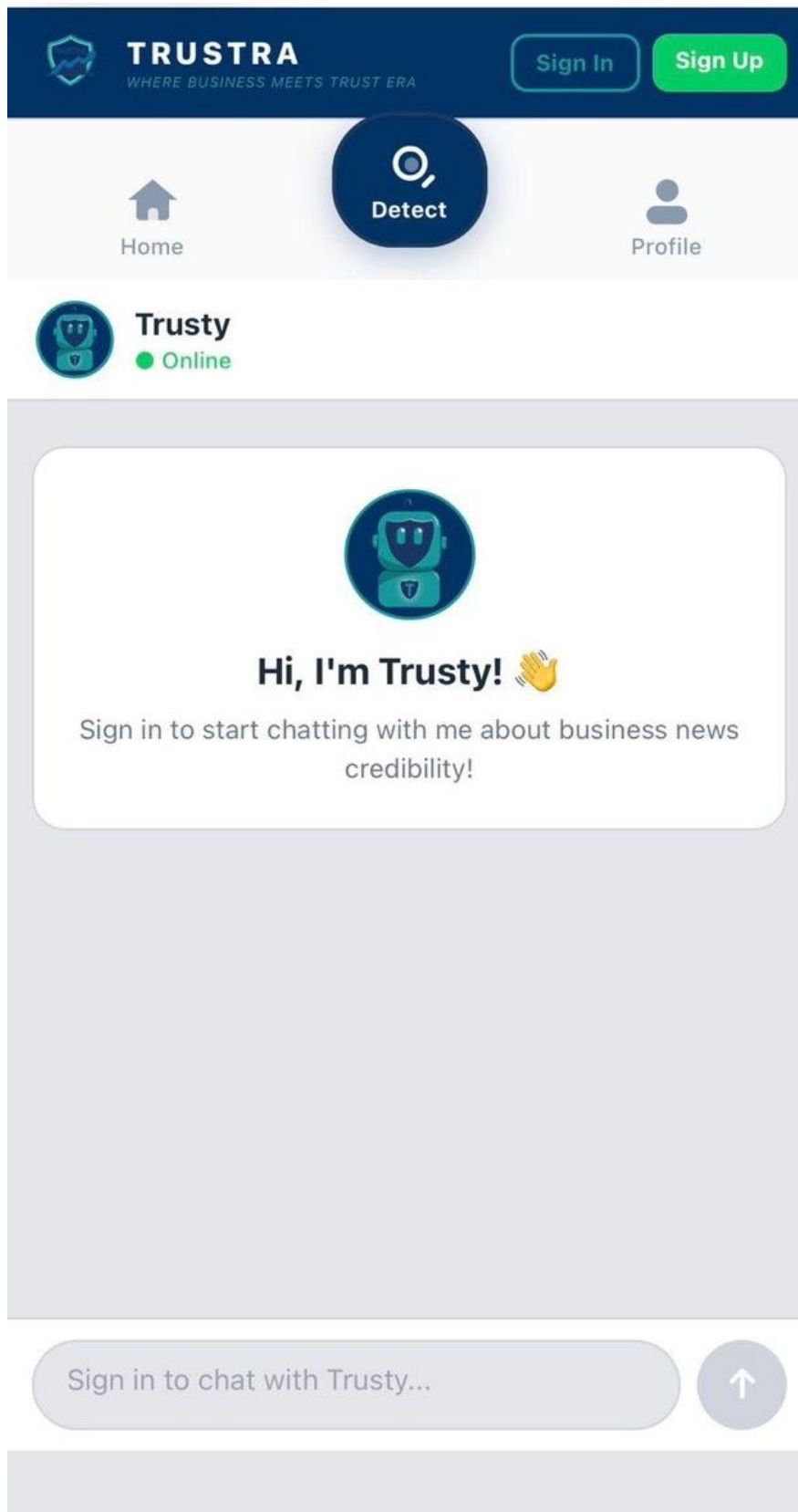


Figure 13: Trusty Screen (Chatbot)

**TRUSTRA**
WHERE BUSINESS MEETS TRUST ERA

Sign InSign Up

Home

Detect

Profile



Create Account

Join Trustra and help fight fake business news

PERSONAL INFORMATION

Full Name
e.g. Ahmed Hassan

Username
e.g. ahmed_h
At least 3 characters.

Email Address
e.g. ahmed@email.com

Phone Number
e.g. +20 1234567890

Birthdate

Trusty

Figure 14: Sign-Up Screen

**TRUSTRA**
WHERE BUSINESS MEETS TRUST ERA

[Sign In](#)[Sign Up](#)


Home


Detect


Profile



Sign In

Welcome back to Trustra

YOUR CREDENTIALS

Email Address

Password
 

[Forgot Password?](#)

Sign In

Don't have an account? [Sign Up](#)


Trusty

Figure 15: Sign-In Screen



Where Business Meets Trust Era

Chapter 4

The Proposed Analytics Tier



This chapter presents the complete technical design and implementation details of Trustra's proposed analytics tier, covering the backend API infrastructure, authentication architecture, frontend integration strategy, AI verification pipeline, and the Trusty business assistant module.

4.1 Data Engineering

4.1.1 Data Engineering Pipeline Using Big Data

Trustra — Big Data Pipeline

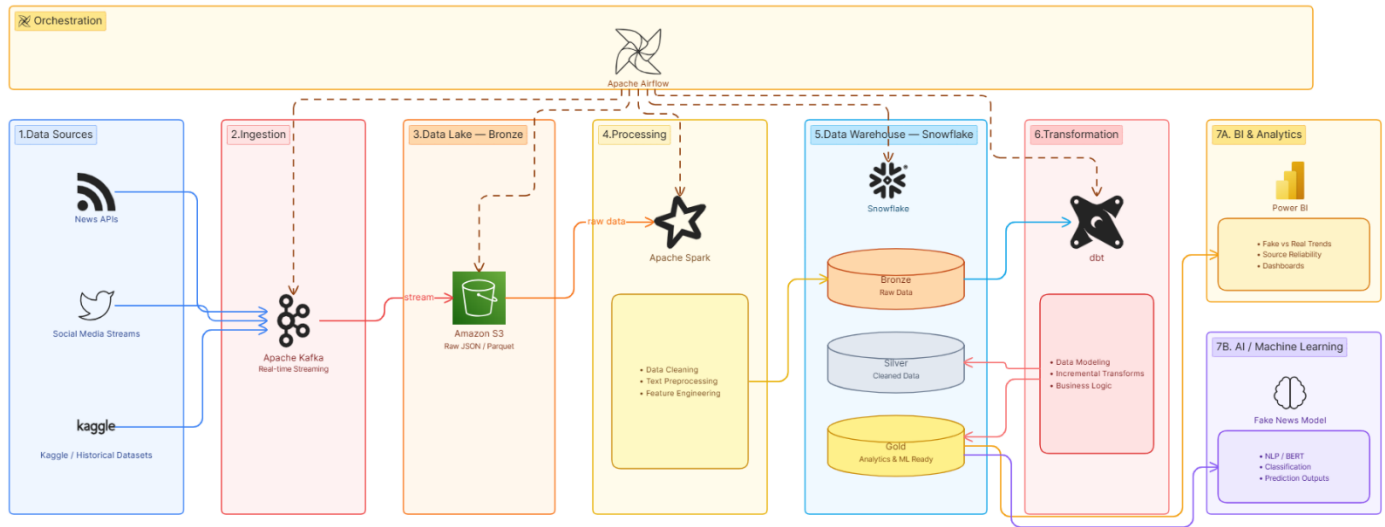


Figure 16: Data Engineering Pipeline

4.1.2 ETL Workflow

The Trustra pipeline follows a structured Extract, Transform, Load process composed of seven sequential stages, fully orchestrated by Apache Airflow.

The Extract phase begins with three categories of data sources: real-time news content pulled from News APIs, social media activity ingested via streaming APIs, and labelled historical datasets sourced from Kaggle benchmarks such as the ISOT Fake News Dataset and the LIAR corpus. All streaming data is routed through Apache Kafka, which decouples producers from consumers and guarantees durable, ordered delivery. Kafka consumers then write raw messages to Amazon S3 in their original JSON or Parquet format, forming the Bronze layer of the data lake — an immutable, append-only archive that serves as the audit trail for the entire pipeline.

The Transform phase begins once Airflow detects a completed S3 partition and triggers an Apache Spark job. Spark applies data cleaning (deduplication, null removal, timestamp normalization, text preprocessing (HTML stripping, tokenisation, named entity recognition), and feature engineering (source credibility scoring, sentiment analysis, engagement velocity calculation). The cleaned and enriched records are written to Snowflake's Silver layer. A second transformation pass is then executed by dbt, which applies business logic, incremental updates, and cross-source joins, promoting records that pass all schema tests to the Gold layer.



The Load phase lands data progressively across Snowflake's three Medallion layers: Bronze holds the raw Snowflake-external-stage copy of S3 data, Silver holds Spark-processed and feature-enriched records, and Gold holds the fully validated, model-ready training corpus. Only records that satisfy all dbt quality gates — including non-null labels, minimum token length, and valid credibility score ranges — are promoted to Gold and made available for model training.

4.1.3 Scalability Discussion

The Trustra pipeline is designed to scale horizontally at every layer without architectural changes.

At the ingestion tier, Apache Kafka scales by adding partitions to existing topics and increasing the number of consumer group members in parallel. Partition keys based on source category ensure even load distribution, while the 7-day retention window allows replay and backfill at scale without data loss.

At the processing tier, Apache Spark runs on a managed cluster (e.g., AWS EMR or Databricks) where executor count and memory are tuned independently per job. The Spark jobs are written with partition-aware transformations so that increasing input volume translates directly to additional parallelism rather than increased per-node pressure. Feature engineering steps that require cross-record comparisons, such as MinHash LSH deduplication, are implemented using Spark's distributed join primitives to avoid bottlenecks.

At the storage tier, Amazon S3 provides virtually unlimited horizontal scaling for the Bronze lake, with no performance degradation as object count grows. Snowflake's compute and storage are fully decoupled, meaning warehouse size can be scaled up for heavy dbt transformation runs and scaled back down for routine loads, keeping costs proportional to workload.

At the orchestration tier, Apache Airflow supports dynamic DAG parallelism and can be deployed on Kubernetes (via the Kubernetes Executor) to spin up isolated pods per task, eliminating shared-resource contention during peak ingestion windows.

The overall pipeline is designed for incremental processing: dbt models are trained incrementally so that only new or updated records are reprocessed on each run, keeping transformation time sub-linear relative to total dataset growth. As the Trustra user base expands and new data sources are onboarded, capacity can be increased at any individual stage without redesigning the pipeline end to end.

4.2 AI/DL

4.2.1 Model Architecture

The system is built on a multi-task learning paradigm using DistilBERT as a shared encoder. Rather than training two independent classifiers, both task heads, fake detection and category classification branch from a single contextual representation produced by the backbone. This design enables knowledge transfer between tasks: learning to identify political, business, and economics language patterns, for example, benefits both the credibility head and the category head simultaneously.

Component	Specification
Backbone	DistilBERT-base-uncased (66M parameters)
Shared representation	[CLS] token — 768-dimensional contextual vector
Shared dropout	Dropout(p=0.3) applied after [CLS] pooling
Fake-news-detection head	Linear(768→256) → GELU → Dropout(0.3) → Linear(256→1) → Sigmoid
Category head	Linear(768→256) → GELU → Dropout(0.3) → Linear(256→N_categories) → Softmax
Output — Task 1	Binary probability: FAKE (0) / TRUE (1)
Output — Task 2	8-class distribution over news categories

Table 5: Model Architecture Overview

Why DistilBERT Over Alternatives

Model	Parameters	Training Speed	Relative Accuracy
CNN-BiLSTM (original)	~5M (from scratch)	Moderate	Baseline
DistilBERT (this work)	66M (pre-trained)	60% faster than BERT	97% of BERT performance
BERT-base	110M (pre-trained)	Slow	Marginal gain over DistilBERT

Table 6: Compare DistilBERT

4.2.2 NLP Pipeline

The dataset originates from the widely-used “Fake and Real News Dataset” published by Clément Bisailon on Kaggle (<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>). The Kaggle source consists of two CSV files: True.csv (real articles scraped from Reuters) and Fake.csv (fabricated articles collected from flagged outlets), totalling approximately 44,000 raw articles.

For this project the two Kaggle files were merged and enriched into a single consolidated file, gold_ml_dataset.csv. The merge added engineered columns (TEXT_LENGTH, TITLE_LENGTH) and a unified

SUBJECT column that maps each article's topic category, enabling the multi-task learning objective. The final consolidated dataset contains 44,269 articles across 7 columns: TITLE, TEXT, SUBJECT, PUBLISH_DATE, LABEL, TEXT_LENGTH, and TITLE_LENGTH. After removing 5,626 duplicate TEXT entries prior to splitting, 38,643 unique articles were used for model training, validation, and testing.

Dataset Statistics

Split	Proportion	Articles	FAKE / TRUE
Training set	70%	27,050	~15,996 / ~15,054
Validation set	15%	5,796	~3,428 / ~3,215
Test set	15%	5,797	~3,428 / ~3,215
Total (post-dedup)	100%	38,643	22,852 FAKE / 21,417 TRUE (raw)

Table 7: Dataset Statistics

Category Labels (8 Classes)

The SUBJECT column provides the category classification head and is never used as a text input feature. The dataset contains 8 subject categories. The categories are: News: 9,050; politics: 6,433; left-news: 4,309; Government News: 1,498; US_News: 783; Middle-east: 778; politicsNews: 11,271; worldnews: 10,145. This clean separation between FAKE subjects and TRUE subjects reflects the original Kaggle source structure.

Data Quality Issues Identified and Resolved

- Duplicate rows: 5,626 duplicate TEXT entries were detected and removed before splitting (from 44,269 raw rows down to 38,643 unique articles) to prevent train/test leakage.
- Spurious header row: a stray row containing the literal string 'title' in the TITLE field (a CSV artefact from the merge) was removed.
- Source-name bias: tokens such as 'r'uters', 'b'eitbart', and 'i'fowars' were perfectly correlated with one label, creating shortcut learning; all were stripped during preprocessing.
- HTML and URL artifacts: appeared exclusively in FAKE articles as scraping noise and were removed.
- Missing PUBLISH_DATE: 44,234 of 44,269 rows (99.9%) have no publication date; this column was dropped entirely and not used as a feature.
- Derived length columns: TEXT_LENGTH and TITLE_LENGTH are engineered columns computed from article content; they correlate with scraping noise and were excluded as input features to prevent data leakage.

4.2.3 Training and Testing

The NLP pipeline transforms raw article text into model-ready token sequences through a series of mandatory cleaning and tokenization steps. The same pipeline is applied identically at training time and inference time to ensure consistency.

Stage 1: Text Cleaning

- Remove Reuters dateline prefix (e.g., 'WASHINGTON (Reuters) -'–'
- Strip all URLs (http / https / www) and raw HTML tags.
- Remove HTML entities (e.g., –) and JavaScript CDATA scraping artifacts.
- Remove source-name tokens: reuters, in62enningsbr62enningsehuffington, daily mail, buzzfeed, vox, the guardian, 21wire, he62enningsenLowercase the entire text.
- Remove bracketed content, punctuation, and numeric tokens.
- Collapse multiple whitespace into single spaces.

Stage 2: Tokenization

- DistilBertTokenizerFast from HuggingFace is used for sub-word tokenization.
- Each article is tokenized with a maximum sequence length of 256 tokens, balancing coverage with training speed.
- Sequences are padded to max_length and truncated if longer.
- Outputs: input_ids tensor and attention_mask tensor passed to the model.

Stage 3: Label Encoding

- Binary fake/true label: 0 = FAKE, 1 = TRUE.

Category label: sklearn LabelEncoder maps the 8 subject strings to integer indices 0–7.

4.2.4 Evaluation Metrics

Training was conducted on a GPU-enabled Google Colab environment using PyTorch with CUDA mixed-precision (AMP) to accelerate computation. The training loop ran for up to 5 epochs with early stopping triggered by the validation fake-news F1 score to avoid overfitting.

Techniques Used

Hyperparameter	Value	Rationale
Max sequence length	256 tokens	Balances content coverage with training speed.
Batch size	32	Standard for BERT-class fine-tuning on GPU.
Learning rate	2e-5	Standard fine-tuning LR for transformer models.
Warmup ratio	0.1 (10% of steps)	Linear warmup prevents early updates from corrupting pre-trained weights.
Gradient clipping	1.0	Prevents exploding gradients during fine-tuning.
Loss weight — Fake head	1.0	Primary task; weighted higher.
Loss weight — Category head	0.3	Secondary task; lower contribution to total loss.

Hyperparameter	Value	Rationale
Early stopping patience	5 epochs	Restores best weights when val fake F1 stops improving.

Table 8: Techniques used in Evaluation Metrics

Overfitting Prevention Techniques

Technique	Effect
Dropout (0.3)	Prevents neuron co-adaptation
Weight Decay (0.01)	L2 regularization on weights
Early Stopping	Restores best generalization point
Gradient Clipping	Stabilizes fine-tuning
LR Warmup + Decay	Protects pre-trained weights
Frozen backbone (Epoch 1)	Stabilizes task-head initialization
WeightedRandomSampler	Corrects class imbalance

Table 9: Overfitting Prevention Techniques

4.2.5 Accuracy Results

Both task heads are evaluated on the held-out test set (15% of the dataset, never seen during training or validation). The following metrics are computed for each head:

Metric	Formula	Interpretation
Accuracy	Correct / Total	Overall fraction of correctly classified articles.
Precision	$TP / (TP + FP)$	Of all articles predicted as a class, how many truly belong to it.
Recall	$TP / (TP + FN)$	Of all articles that truly belong to a class, how many were retrieved.
F1 Score (macro)	$2 \cdot P \cdot R / (P + R)$	Harmonic mean of precision and recall, averaged equally across classes.
Confusion Matrix	N×N count grid	Cell [i][j] shows how many class-i Ircticles were predicted as class-j.

Table 10: Model Accuracy Results

For the fake-news head the macro F1 score is treated as the primary metric, as it is sensitive to both precision and recall and handles any remaining class imbalance. For the category head, per-class metrics reveal which news topics are harder to classify.

4.2.6 Confusion Metrix

The model was evaluated on the held-out test set after 5 epochs of training. The results demonstrate near-perfect performance on the primary fake detection task and strong performance on the multi-class category classification task.

Fake News Detection — Test Set Performance

Class	Precision	Recall	F1 Score	Support
FAKE	~99%	99.5%	99.7%	2,618
TRUE	~99%	99.8%	99.7%	3,179
Overall Accuracy	99.6%			5,797
Macro F1	99.6%			

Table 11: Fake News Detection — Test Set Performance

Category Classification — Test Set Performance

Category	Precision	Recall	F1 Score	Support
Government News	50%	57%	53%	88
Middle-east	78%	82%	80%	55
News	89%	97%	93%	1,331
US_News	95%	71%	81%	58
left-news	91%	17%	29%	124
politics	79%	86%	82%	952
politicsNews	99%	99%	99%	1,633
worldnews	98%	99%	99%	1,556
Overall Accuracy	~93.5%			5,797

Table 12: Category Classification — Test Set Performance

Government News and left-news show lower F1 scores, primarily due to class imbalance, their smaller sample sizes, and semantic overlap with other political categories. These are the main areas targeted by future improvements.

4.2.7 Training Curves



The training curves below illustrate the model's learning dynamics across 5 epochs. Three metrics are tracked: total combined loss, fake detection accuracy, and category classification accuracy, each plotted separately for the training and validation sets.

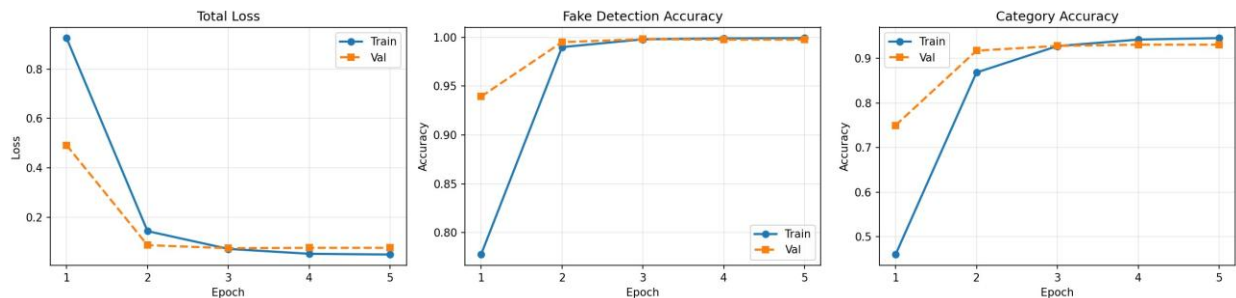


Figure 17: Training and validation curves over 5 epochs — Total Loss (left), Fake Detection Accuracy (center), Category Accuracy (right)

Key observations from the training curves:

- Total loss drops sharply after epoch 1 (from ~0.90 to ~0.15) and plateaus by epoch 3, indicating rapid and stable convergence.
- Validation loss starts lower than training loss and tracks closely, confirming the model is not overfitting.
- Fake detection accuracy reaches 99%+ on both train and validation sets by epoch 3 and remains stable.
- Category accuracy climbs steadily from ~75% at epoch 1 to ~93% by epoch 5, with train and validation curves nearly coinciding — a sign of good generalization.

4.2.8 Confusion Matrix

Confusion matrices visualize the detailed prediction behaviour of both task heads on the test set. The left matrix shows fake news detection (binary) and the right shows category classification (8-class).

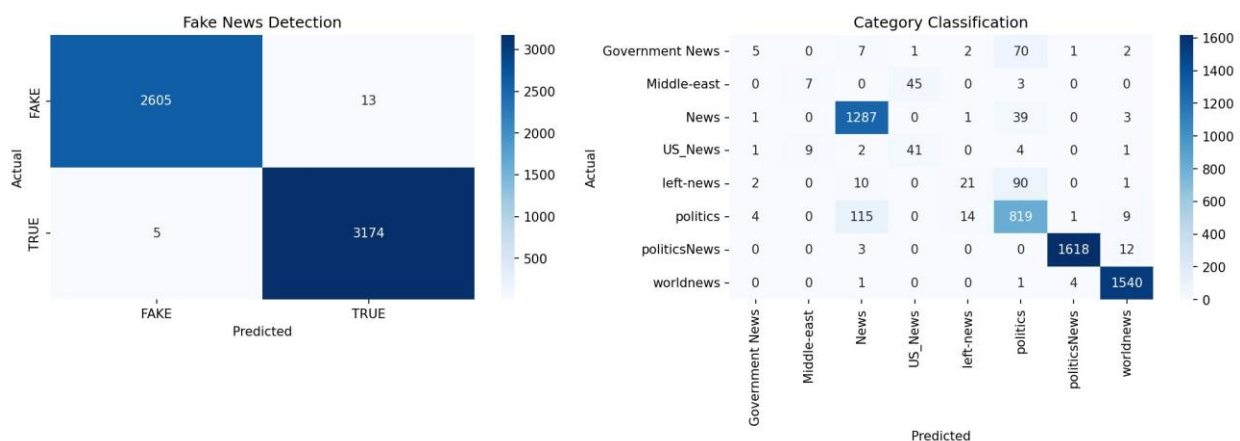


Figure 18: Confusion matrices for Fake News Detection (left) and Category Classification (right) on the held-out test set

Fake News Detection Interpretation

- 2,605 FAKE articles correctly identified; only 13 misclassified as TRUE (false negatives, 0.5% error rate).
- 3,174 TRUE articles correctly identified; only 5 misclassified as FAKE (false positives, 0.2% error rate).
- The asymmetric error pattern is desirable: labelling real news as fake (FP) is rarer than missing fake news (FN).

Category Classification Interpretation

- politicsNews (1,618 correct, 12 errors) and worldnews (1,540 correct, 6 errors) are classified almost perfectly.
- News (1,287 correct) achieves very high recall with minor confusion toward politics.
- politics (819 correct) shows some confusion with News and left-news, reflecting linguistic overlap between these categories.
- Government News (70 out of 88 predicted as politics) is the weakest category, consistent with its small support and stylistic similarity to political reporting.
- left-news (21 correct, 90 misclassified as politics) has the lowest recall, suggesting this category is linguistically very close to the broader politics category.

4.3 Backend API Implementation

4.3.1 Authentication System

Overview & Technology Stack

The Trustra backend is a RESTful API service built to support an AI-powered fake-news verification platform. It is hosted on Railway (production) at:

<https://fakenews-backend-production.up.railway.app>

Local development runs on <http://localhost:5000>. The API follows a resource-oriented design where every response — success or failure — conforms to a single consistent envelope format, ensuring that frontend clients can parse responses uniformly without special-casing individual endpoints.

Core Architectural Decisions

- All requests use Content-Type: application/json — no multipart form data except for avatar uploads (transmitted as base64 strings within JSON).
- Every success response carries a success: true flag and a data object; error responses carry success: false, a human-readable message, and an optional errors array for field-level validation failures.



- Paginated endpoints append a meta block containing currentPage, totalPages, totalItems, itemsPerPage, hasNextPage, and hasPrevPage, allowing the frontend to implement infinite scroll or traditional pagination without any additional requests.
- HTTP status codes are used semantically: 202 Accepted is returned for asynchronous AI verification tasks; 409 Conflict for duplicate resources; 422 Unprocessable Entity for validation failures; and 429 Too Many Requests for rate-limited endpoints.

4.3.2 Endpoint Architecture

The API is organized into five functional modules. Public endpoints are accessible without authentication and form the read-heavy portion of the platform, while protected endpoints require a valid JWT bearer token.

Public Endpoints

Method	Endpoint / Description
GET	/api/health — Server health check
POST	/api/auth/signup — Register new user account
POST	/api/auth/signin — Authenticate and receive tokens
POST	/api/auth/refresh-token — Obtain new access/refresh token pair
GET	/api/community/categories — Retrieve all available news categories
GET	/api/community/feed — Browse community-shared verified news
GET	/api/news/:id — Fetch a single news item (includes AI result)
GET	/api/users/search?username= — Search users by username prefix
GET	/api/users/:username — View a user's public profile and shared news

Table 13: Public Endpoints

Protected Endpoints (Authorization: Bearer <token> required)

Method	Endpoint / Description
POST	/api/news — Submit article for AI verification
GET	/api/news/my-news — List articles submitted by current user

Method	Endpoint / Description
GET	/api/news/saved — List articles saved by current user
DELETE	/api/news/:id — Delete own article
POST	/api/news/:id/share — Share verified real article to community feed
DELETE	/api/news/:id/share — Remove article from community feed
POST	/api/news/:id/like — Toggle like on a verified real article
POST	/api/news/:id/save — Toggle save on a verified real article
GET	/api/auth/me — Retrieve current user profile
POST	/api/auth/logout — Invalidate session tokens
PATCH	/api/users/me/profile — Update display name or username
PATCH	/api/users/me/avatar — Upload new profile picture (base64)
PATCH	/api/users/me/password — Change account password
POST	/api/trusty/chat — Send message to Trusty AI assistant
DELETE	/api/trusty/chat/history — Clear Trusty conversation history

Table 14: Protected Endpoints

4.3.3 News Verification Pipeline

The most technically complex part of the backend is the asynchronous AI verification flow. When a user submits an article, the backend immediately acknowledges the request with HTTP 202 Accepted while the AI model processes the content in the background. The news item is persisted in the database with `verification_status` set to 'pending'.

Submission Flow

1. Client sends POST /api/news with title and content fields.
2. Backend saves the record with `verification_status`: 'pending' and category: 'other' (temporary).

3. 202 Accepted response is returned immediately containing the news ID.
4. AI model analyzes the article and updates both verification_status ('real' or 'fake') and category (e.g., 'health', 'politics') in the database.
5. Frontend polls GET /api/news/:id every 3–5 seconds until status is no longer 'pending' typically within 5–15 seconds).

Business Rules Enforced by the API

- Only articles with verification_status = 'real' may be shared, liked, or saved.
- Any authenticated user may share any verified real article — not only the submitter.
- Each user may share a given article only once (enforced at database level, returns 409 on duplicate).
- Like and save endpoints are idempotent toggles: a second call on the same resource reverses the action.
- Deletion cascades: removing an article also removes all associated likes, saves, and shares.

Note on Category Assignment

Users do not supply a category when submitting news. The AI model determines both the real/fake classification and the content category simultaneously. This removes human bias from categorization and ensures categories are assigned consistently across the platform.

Supported categories: politics, business, health, science, technology, sports, entertainment, economy, world, other

4.3.4 Rate Limiting Strategy

Three distinct rate-limit tiers protect the service from abuse and manage AI inference costs:

Endpoint Group	Limit	Window
General API	100 requests	Per 15 minutes
AI endpoints (news/trusty)	5 requests	Per 1 minute
Auth endpoints	10 attempts	Per 15 minutes

Table 15: Rate Limiting Strategy

Exceeding a limit returns HTTP 429 with success: false and a human-readable retry message. AI endpoints are most tightly restricted because each request incurs external model inference costs.

4.3.5 Authentication System

A) Token Architecture

Trustra uses a dual-token JWT strategy consisting of a short-lived access token and a longer-lived refresh token. This pattern balances security (tokens expire and can be revoked) with usability (users are not forced to re-authenticate frequently).

Token Type	Purpose	Lifetime	Storage Advice
Access Token	Authorise every API request	7 days	Secure in-memory or encrypted storage
Refresh Token	Obtain a new access token pair	Long-lived	Secure persistent storage (keychain, etc.)

Table 16: Token Architecture

Tokens are passed via the Authorization HTTP header using the Bearer scheme:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

B) Registration & Validation Rules

User registration (POST /api/auth/signup) enforces the following server-side constraints:

Field	Constraints
fullName	Required, 2–100 characters
username	Required, 3–30 chars, lowercase letters / numbers / underscores only
email	Required, valid email format, globally unique
phoneNumber	Required, international format with country code (e.g. +201012345678)
birthdate	Required, YYYY-MM-DD format, user must be at least 13 years old
password	Required, minimum 8 characters, must include uppercase letter and number
passwordConfirmation	Required, must exactly match password field

Table 17: Registration & Validation Rules



C) Token Lifecycle & Refresh Flow

The recommended client-side implementation uses an HTTP interceptor that transparently handles token expiry:

```
// Pseudocode — HTTP interceptor
async function apiRequest(url, options) {
  try {
    return await fetch(url, withAuth(options));
  } catch (err) {
    if (err.status === 401 && err.message === 'Token expired') {
      const newTokens = await refreshTokens();
      if (newTokens) {
        saveTokens(newTokens);
        return await fetch(url, withAuth(options)); // retry
      }
      redirectToLogin();
    }
    throw err;
  }
}
```

If the refresh endpoint itself returns 401 (invalid or expired refresh token), the client must clear all stored tokens and redirect the user to the login screen. This prevents an infinite refresh loop.

Key Security Behaviours

- Logout (POST /api/auth/logout) invalidates the refresh token server-side.
- Refresh token mismatch returns 401 with message 'Refresh token mismatch. Please log in again.'
 - this protects against token theft in rotation-based systems.



- The GET /api/auth/me endpoint validates the access token and returns 401 if the user account no longer exists in the database, even with a structurally valid token.

D) Avatar & Profile Updates

Profile picture uploads are handled as base64-encoded data URLs transmitted inside a standard JSON body. This avoids multipart form complexity at the cost of payload size. The backend accepts PNG, JPG, JPEG, GIF, and WEBP formats.

```
PATCH /api/users/me/avatar
Content-Type: application/json
Authorization: Bearer <token>

{
  "avatar": "data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAA..."
}
```

Frontend clients should compress images to under 500 KB before base64 encoding to avoid excessively large payloads. The `FileReader.readAsDataURL()` API can be used in browser environments; mobile clients should use platform-native base64 encoding libraries.

4.3.6 Integration with Frontend, Trusty and AI Model

A) Frontend Integration

Community Feed

The community feed (GET /api/community/feed) is the platform's primary discovery surface. It is publicly accessible — guests may browse without authentication — and becomes richer when a valid access token is supplied. Authenticated responses include per-article `is_liked` and `is_saved` boolean fields derived from the requesting user's interaction history, enabling the frontend to render correct toggle states without additional requests.

Category filtering and full-text search are composable query parameters:

```
GET /api/community/feed?category=health&search=vaccine&page=1&limit=20
```

```
// Response includes meta.activeCategory and meta.availableCategories
```

```
// for dynamic tab rendering without a separate categories request
```

Guest-vs-Authenticated Behaviour

The `isGuest` flag in the feed response acts as a frontend capability signal: when true, any interaction attempt (like, save, share) should be intercepted and replaced with a login prompt rather than an API call. This eliminates the need for the frontend to maintain its own authentication state just to gate UI elements.

Avatar Display

Avatar data is returned as a raw base64 data URL string in all user objects where avatar is not null. This value can be used directly as the `src` attribute of an HTML `<img>` element or equivalent in mobile frameworks, removing the need for a separate image hosting service.

Pagination

All list endpoints accept `page` and `limit` query parameters (default: 10, maximum: 50). The `meta.hasNextPage` boolean allows simple load-more or infinite scroll implementations. Clients should render a 'Load More' button only when `hasNextPage` is true, and disable it once false.

B) Integration with the AI Verification Model

The AI model integration is fully server-side and transparent to the frontend. Clients interact only with the standard REST endpoints; the model is invoked internally by the backend on news submission. The model performs two tasks concurrently:

- **Classification:** determines whether the submitted article is real or fake news.
- **Categorisation:** assigns one of ten predefined categories (politics, business, health, science, technology, sports, entertainment, economy, world, other).

Because model inference is not instantaneous, the backend decouples submission from result delivery using an asynchronous processing pattern. The client is responsible for polling the single-article endpoint at a 3–5 second interval until the status field transitions from 'pending' to either 'real' or 'fake'.



```
// Frontend polling pseudocode
async function pollVerification(newsId) {
  const INTERVAL_MS = 3000;
  while (true) {
    const { data } = await api.get(`/api/news/${newsId}`);
    const status = data.news.verification_status;
    if (status === 'real') { showRealBadge(); break; }
    if (status === 'fake') { showFakeBadge(); break; }
    await sleep(INTERVAL_MS);
  }
}
```

AI Rate Limiting Note

The AI submission endpoint (`/api/news`) is rate-limited to 5 requests per minute per user. Frontend implementations should disable the 'Submit' button after a successful submission and re-enable it only after the verification result is received, naturally preventing users from hitting the rate limit during normal usage.

D) Integration with Trusty AI Business Assistant

Trusty is a scoped AI chatbot embedded in the platform. It is designed exclusively for business topics — business planning, marketing strategies, financial modelling, and entrepreneurship advice — and will decline questions outside this domain. It is backed by a persistent, per-user conversation store.

Conversation Persistence

Unlike typical stateless AI API wrappers, Trusty maintains full conversation history in the database and associates it with the authenticated user's account. This means:

- Conversation context is preserved across browser sessions, app restarts, and even logout/login cycles.
- Each user's conversation is isolated — users cannot see or influence each other's chat histories.

- The frontend does not need to manage or transmit conversation history with each request; the backend reconstructs it automatically.

API Contract

POST /api/trusty/chat

Authorization: Bearer <token>

Request: { "message": "Help me create a marketing plan for a SaaS product" }

Response: { "reply": "...", "historyLength": 4 }

// historyLength: number of conversation turns so far

// Use this to display a message count or warn user before clearing history

History Management

Provide users with a visible 'Clear Conversation' control that calls DELETE /api/trusty/chat/history. This is especially important for privacy-conscious users and for cases where a user wants to start a new business inquiry from a clean context without the AI being influenced by prior sessions.

Constraint	Value
Message max length	2,000 characters per request
Rate limit	5 requests per minute
Topic scope	Business, marketing, finance, entrepreneurship only
History storage	Persistent per user, cross-session

Table 18: History Management

4.3.7 Additional Technical Considerations

A) Error Handling Architecture

The API enforces a uniform error response schema across all endpoints, making centralised error handling straightforward for frontend clients. Every error response takes one of three forms:

```
{
  "success": false,
  "message": "Human-readable description of the error"
}
```

Validation Error

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "email", "message": "Please provide a valid email" },
    { "field": "password", "message": "Password must be at least 8 characters" }
  ]
}
```

The errors array enables the frontend to associate validation messages directly with their corresponding form fields without string parsing. Validation errors return HTTP 422; structural or business-logic errors return 400, 401, 403, 404, or 409 depending on the violation type.

HTTP Code	Meaning	Recommended Frontend Response
400	Bad request	Show the error message to the user
401	Unauthorised	Attempt token refresh; redirect to login if it fails
403	Forbidden	Show 'You do not have permission' and do not retry
404	Not found	Show a not-found screen or remove the item from local state
409	Conflict	Show field-level error (e.g., 'Username already taken')

422	Validation	Iterate errors array and annotate individual form fields
429	Rate limit	Disable the triggering button temporarily; show a wait message
500	Server error	Show a generic retry message; do not display raw error to users

Table 19: Errors Array

B) Data Model Reference

Two primary data models are returned by multiple endpoints. Understanding their shape allows frontend components to be written generically against these structures.

User Object

```
{
  "id": 1,
  "full_name": "John Doe",
  "username": "johndoe",
  "email": "john@example.com",
  "phone_number": "+201012345678",
  "birthdate": "1995-03-15",
  "avatar": null,          // null OR base64 data URL string
  "created_at": "2025-01-01T00:00:00.000Z",
  "updated_at": "2025-01-01T00:00:00.000Z"
}
```

News Object

```
{
  "id": 1,
```

```
"title": "Article headline",
"content": "Full article body...",
"verification_status": "real", // "pending" | "real" | "fake"
"verified_at": "2025-01-01T...", // null while pending
"submitted_by": 1,           // user ID of submitter
"category": "health",
"likes_count": 10,
"saves_count": 5,
"shares_count": 3,
"is_liked": true,           // only present when authenticated
"is_saved": false,         // only present when authenticated
"author": { /* User object without sensitive fields */ },
"shares": [ /* array of sharing user objects */ ]
}
```

D) Security Considerations

- All sensitive fields (email, phone number, birthdate) are excluded from public profile responses (GET /api/users/:username) — only id, full_name, username, avatar, and created_at are exposed publicly.
- Password change requires supplying the current password before accepting the new one, preventing account takeover if an access token is compromised.
- Ownership validation is performed server-side on all destructive operations: only the submitter of a news article may delete it; cross-user deletion attempts return 403 Forbidden.
- The avatar endpoint validates that the incoming string is a genuine base64-encoded image and rejects arbitrary binary blobs, reducing the attack surface for payload injection.
- Rate limiting on auth endpoints (10 attempts / 15 minutes) provides basic brute-force protection for the login and registration flows.

E) Scalability & Deployment Notes

The current deployment on Railway provides managed infrastructure with auto-scaling capabilities. Several design decisions in the API support horizontal scaling:



- JWT tokens are stateless and self-contained — any server instance can validate them without a shared session store.
- The asynchronous AI verification pattern decouples compute-intensive inference from request handling, allowing the API tier to scale independently of the AI model tier.
- Pagination limits (max 50 items per request) bound the maximum response payload size, protecting both server memory and client rendering performance.
- The persistent Trusty conversation history is stored in the database rather than in server memory, making the chat service stateless at the application tier and safe to run behind a load balancer with multiple instances.

Production Base URL

`https://fakenews-backend-production.up.railway.app`

Local Development Base URL

`http://localhost:5000`

All requests require Content-Type: application/json.

4.4 Frontend Implementation

The frontend implementation of Trustra follows a modular architecture where screens, reusable components, and API services are separated into organized folders. The application was developed using React Native with Expo to support both Android and iOS platforms efficiently.

The application communicates with the backend using REST APIs. All API requests are centralized in a dedicated service file (`api.js`), which manages authentication tokens, request handling, and automatic token refresh when sessions expire.

A) Pages and Components

- **HomeScreen:** Displays the community news feed with categories, search, and interaction features.
- **SignUpScreen:** Allows users to create a new account with validation checks.
- **SignInScreen:** Handles user login and authentication.
- **ModelScreen:** Implements the AI fake news detection feature.
- **ChatbotScreen:** Provides the Trusty AI chatbot for business news verification.
- **ProfileScreen:** Shows user profile information, statistics, and activity.
- **Navbar Component:** Reusable navigation bar with bottom navigation and authentication buttons.

B) Responsive Frontend

The frontend was designed to be responsive across different mobile screen sizes. Flexible layouts, scalable spacing, and reusable styles were used to maintain consistent user experience on multiple devices. React Native's flexbox layout system was used extensively to ensure proper alignment and adaptability. Scroll views and responsive cards were implemented to improve usability on smaller screens.

C) User Handling

User authentication is implemented using JWT tokens. During login, access and refresh tokens are stored locally using AsyncStorage. The application automatically verifies stored tokens during startup to maintain persistent login sessions.

The frontend also includes validation mechanisms to ensure users enter valid information during registration and authentication processes.

D) Additional Technical Features

Automatic token refresh when authentication expires.

Optimistic UI updates for likes and interactions.

AI-powered fake news verification with polling mechanism.

Integrated chatbot assistant for business news support.

Pull-to-refresh and pagination for improved feed performance.

Custom-designed Trustra branding and reusable design system.

4.4.1 Code Samples

```
C:\Users> ahmed> trustra> app> index.tsx > ...
1  import React, { useState, useEffect } from 'react';
2  import { View, ActivityIndicator, StyleSheet } from 'react-native';
3  import AsyncStorage from '@react-native-async-storage/async-storage';
4  import HomeScreen from './HomeScreen';
5  import SignUpScreen from './SignUpScreen';
6  import SignInScreen from './SignInScreen';
7  import ChatbotScreen from './ChatbotScreen';
8  import ModelScreen from './ModelScreen';
9  import ProfileScreen from './ProfileScreen';
10 import { authAPI, getAccessToken, clearTokens } from '../api/api';
11 // File: app/index.tsx
12 //
13 // Handles:
14 // - Auto-login check on app start (reads saved token)
15 // - Global auth state (isLoggedIn, currentUser)
16 // - Screen navigation
17 // - Passing login/logout callbacks to screens
18 // -----
19
20 export default function Index() {
21   const [screen, setScreen] = useState('Home');
22   const [isLoggedIn, setIsLoggedIn] = useState(false);
23   const [currentUser, setCurrentUser] = useState(null);
24   const [checkingAuth, setCheckingAuth] = useState(true);
25
26   // — Check if user is already logged in on app start —————
27   useEffect(() => {
28     const checkAuth = async () => {
29       try {
30         const token = await getAccessToken();
31         if (token) {
32           // Token exists – verify it by fetching current user
33           const data = await authAPI.getMe();
34           setCurrentUser(data.user || data);
35           setIsLoggedIn(true);
36         }
37       } catch {
38         // Token not found or invalid, user will be redirected to login
39       }
40     };
41     checkAuth();
42   }, []);
43 }
```

Figure 19: Code sample-1

```

C: > Users > ahmed > trustra > app > JS SignInScreen.js > ...
1  import React, { useState, useRef } from 'react';
2  import {
3    View, Text, TextInput, TouchableOpacity, StyleSheet,
4    ScrollView, SafeAreaView, StatusBar, Modal, ActivityIndicator,
5  } from 'react-native';
6  import NavBar, { TrustyButton } from '../components/NavBar';
7  import { authAPI } from '../api/api';
8
9  // -----
10 // TRUSTRA - Sign In Screen (with OTP verification)
11 // File: app/SignInScreen.js
12 // -----
13
14 const C = {
15   primary: '#003366', secondary: '#00CC66', accent: '#009999',
16   warning: '#CC0033', background: '#E5E7EB', text: '#1C2B3A',
17   muted: '#6B7A8D', card: '#FFFFFF', border: '#D1D5DB',
18 };
19
20 const ForgotModal = ({ visible, onClose }) => {
21   const [fpEmail, setFpEmail] = useState('');
22   const [fpSent, setFpSent] = useState(false);
23   const [fpError, setFpError] = useState('');
24
25   const handleSend = () => {
26     if (!/^[a-zA-Z0-9+@.\\$%&'*/~`{|}~\s+$/i.test(fpEmail)) { setFpError('Enter a valid email address.');
27     setFpSent(true); setFpError('');
28   };
29   const handleClose = () => { setFpEmail(''); setFpSent(false); setFpError(''); onClose(); };
30
31   return (
32     <Modal visible={visible} animationType="slide" transparent>
33       <View style={styles.modalOverlay}>
34         <View style={styles.modalBox}>
35           <Text style={styles.modalTitle}>Forgot Password?</Text>
36           {fpSent ? (

```

Figure 20: Code sample-2

```

C: > Users > ahmed > trustra > api > JS api.js > ...
1  import AsyncStorage from '@react-native-async-storage/async-storage';
2
3  // -----
4  // TRUSTRA - Central API Service
5  // File: api/api.js
6  // Base URL: https://trustera-backend-production.up.railway.app
7  // -----
8
9  const BASE_URL = 'https://trustera-backend-production.up.railway.app';
10
11 // --- Request timeout (ms) ---
12 const TIMEOUT_MS = 20000; // 20 seconds
13
14 // --- Token helpers ---
15 export const saveTokens = async (accessToken, refreshToken) => {
16   await AsyncStorage.setItem('accessToken', accessToken);
17   if (refreshToken) await AsyncStorage.setItem('refreshToken', refreshToken);
18 };
19
20 export const getAccessToken = async () => {
21   return await AsyncStorage.getItem('accessToken');
22 };
23
24 export const getRefreshToken = async () => {
25   return await AsyncStorage.getItem('refreshToken');
26 };
27
28 export const clearTokens = async () => {
29   await AsyncStorage.removeItem('accessToken');
30   await AsyncStorage.removeItem('refreshToken');
31 };
32
33 // --- Fetch with timeout ---
34 const fetchWithTimeout = (url, options, timeoutMs = TIMEOUT_MS) => {
35   const controller = new AbortController();
36   const timeoutId = setTimeout(() => controller.abort(), timeoutMs);

```

Figure 21: Code sample-3

```

C:\Users> ahmed > trustra > app > JS ProfileScreen.js > ...
1  import React, { useState, useEffect } from 'react';
2  import {
3    View, Text, TextInput, TouchableOpacity, StyleSheet,
4    ScrollView, SafeAreaView, StatusBar, Modal, ActivityIndicator,
5    Image, FlatList,
6  } from 'react-native';
7  import Navbar, { TrustyButton } from '../components/Navbar';
8  import { authAPI, newsAPI, usersAPI } from '../api/api';
9
10 // -----
11 // TRUSTRA - Profile Screen
12 // File: app/ProfileScreen.js
13 // -----
14
15 const C = {
16   primary: '#003366', secondary: '#00CC66', accent: '#009999',
17   warning: '#CC0033', background: '#E5E7EB', text: '#1C2B3A',
18   muted: '#6B7A8D', card: '#FFFFFF', border: '#D1D5DB',
19 };
20
21 const toStr = (val) => {
22   if (val == null) return '';
23   if (typeof val === 'string') return val;
24   if (typeof val === 'number') return String(val);
25   if (typeof val === 'object') return '';
26   return String(val);
27 };
28

```

Figure 22: Code sample-4

```

C:\Users> ahmed > trustra > app > JS ModelScreen.js > ...
1  import React, { useState, useRef } from 'react';
2  import {
3    View, Text, TextInput, TouchableOpacity, StyleSheet,
4    ScrollView, SafeAreaView, StatusBar, Modal, ActivityIndicator,
5    Animated, Share, KeyboardAvoidingView, Platform,
6  } from 'react-native';
7  import Navbar, { TrustyButton } from '../components/Navbar';
8  import { newsAPI } from '../api/api';
9
10 // -----
11 // TRUSTRA - Model Screen (API Connected with polling)
12 // File: app/ModelScreen.js
13 // -----
14
15 const C = {
16   primary: '#003366', secondary: '#00CC66', accent: '#009999',
17   warning: '#CC0033', background: '#E5E7EB', text: '#1C2B3A',
18   muted: '#6B7A8D', card: '#FFFFFF', border: '#D1D5DB',
19 };
20
21 const pollNewsResult = (newsId, onResult, onError) => {
22   let attempts = 0;
23   const maxAttempts = 20;
24   const poll = async () => {
25     attempts++;
26     try {
27       const data = await newsAPI.getNews(newsId);
28       const status = (data.verification_status || data.status || '').toLowerCase();
29       if (status === 'real' || status === 'verified') {
30         onResult({ verdict: 'REAL', data });
31       } else if (status === 'fake') {
32         onResult({ verdict: 'FAKE', data });
33       } else if (attempts >= maxAttempts) {
34         onError('Analysis timed out. Please try again.');
```

Figure 23: Code sample-5

```

C:\Users> ahmed > trustra > app > JS SignUpScreen.js > ...
1  import React, { useState, useRef } from 'react';
2  import {
3    View, Text, TextInput, TouchableOpacity, StyleSheet,
4    ScrollView, SafeAreaView, StatusBar, Modal, ActivityIndicator, Platform,
5  } from 'react-native';
6  import DateTimePicker from '@react-native-community/datetimepicker';
7  import NavBar, { TrustyButton } from '../components/NavBar';
8  import { authAPI } from '../api/api';
9
10 // -----
11 // TRUSTRA – Sign Up Screen (with OTP email verification)
12 // File: app/SignUpScreen.js
13 // -----
14
15 const C = {
16   primary: '■ #003366', secondary: '■ #00CC66', accent: '■ #009999',
17   warning: '■ #CC0033', background: '■ #E5E7EB', text: '■ #1C2B3A',
18   muted: '■ #6B7A8D', card: '■ #FFFFFF', border: '■ #D1D5DB',
19 };
20
21 const validate = (fields) => {
22   const errors = {};
23   if (!fields.fullName.trim()) errors.fullName = 'Full name is required.';
24   if (!fields.username.trim()) errors.username = 'Username is required.';
25   else if (fields.username.length < 3) errors.username = 'Username must be at least 3 characters.';
26   if (!fields.email.trim()) errors.email = 'Email is required.';
27   else if (!/^[a-zA-Z0-9+@-]+\.[a-zA-Z0-9+@-]+\.[a-zA-Z0-9+@-]+$/i.test(fields.email)) errors.email = 'Enter a valid email address.';
28   if (!fields.phoneNumber.trim()) errors.phoneNumber = 'Phone number is required.';
29   if (!fields.birthdate) errors.birthdate = 'Birthdate is required.';
30   if (!fields.password) errors.password = 'Password is required.';
31   else if (fields.password.length < 8) errors.password = 'Password must be at least 8 characters.';
32   if (!fields.passwordConfirmation) errors.passwordConfirmation = 'Please confirm your password.';
33   else if (fields.password !== fields.passwordConfirmation) errors.passwordConfirmation = 'Passwords do not match.';
34   if (!fields.agreed) errors.agreed = 'You must agree to the Terms & Conditions.';
35   return errors;
36 };

```

Figure 24: Code sample-6

```

C:\Users> ahmed > trustra > app > JS ChatbotScreen.js > ...
1  import React, { useState, useRef, useEffect } from 'react';
2  import {
3    View, Text, TextInput, TouchableOpacity, StyleSheet,
4    FlatList, SafeAreaView, StatusBar, KeyboardAvoidingView,
5    Platform, Modal, ActivityIndicator, Image,
6  } from 'react-native';
7  import NavBar from '../components/NavBar';
8  import { trustyAPI } from '../api/api';
9
10 // -----
11 // TRUSTRA – Trusty Chatbot Screen (API Connected)
12 // File: app/ChatbotScreen.js
13 // -----
14
15 const C = {
16   primary: '■ #003366', secondary: '■ #00CC66', accent: '■ #009999',
17   warning: '■ #CC0033', background: '■ #E5E7EB', text: '■ #1C2B3A',
18   muted: '■ #6B7A8D', card: '■ #FFFFFF', border: '■ #D1D5DB',
19 };
20
21 const SUGGESTED_QUESTIONS = [
22   '🔍 How can I tell if a business news story is fake?',
23   '📰 What are common signs of financial misinformation?',
24   '🏢 How do I verify a company\'s financial claims?',
25   '📰 Which sources are most reliable for business news?',
26   '⚠️ What tactics do fake news sites use?',
27   '🤖 How does AI help detect fake news?',
28 ];
29
30 const TrustyAvatar = ({ size = 32 }) => (
31   <View style={[avStyle.wrap, { width: size, height: size, borderRadius: size / 2 }]}>
32     <Image
33       source={require('../assets/trusty_no_background.png')}
34       style={{ width: size * 0.85, height: size * 0.85 }}
35       resizeMode="contain"
36     />
37   </View>

```

Figure 25: Code sample-7



Where Business Meets Trust Era

Chapter 5

Conclusion and Future Work



5.1 Data Engineering Limitations and Future Improvement

A) Big Data Limitations

Labeled Data Scarcity The availability of high-quality, labelled training data remains one of the most fundamental constraints in the pipeline. The current pipeline compensates through semi-supervised labelling — applying a confidence-thresholded classifier to promote unlabelled Silver records to the Gold layer — but this introduces a risk of label noise propagating into the training corpus and reducing model reliability.

Concept Drift and Model Staleness The language, framing, and distribution of fake business news evolve continuously. A model trained on historical data can degrade in accuracy as incoming articles diverge from the training distribution. The current pipeline triggers retraining on a weekly schedule or when the Gold layer record delta crosses a threshold, but this approach is reactive rather than proactive, meaning performance degradation may go undetected between retraining cycles.

Processing Latency in the Spark Layer Apache Spark Structured Streaming operates on a micro-batch model, which introduces an inherent latency floor between data arrival and Gold layer availability. As a result, a newly published article may take several minutes to traverse the full Bronze-to-Gold pipeline before it is available for credibility assessment or model inference, limiting the system's responsiveness to breaking news events.

Schema Drift and Pipeline Fragility News APIs and social media platforms frequently alter their response schemas — adding, renaming, or deprecating fields — without prior notice. Such changes can silently corrupt Bronze records or break downstream Spark parsing logic, causing data gaps that only surface during dbt testing at the Gold layer, by which point a meaningful volume of records may already be affected.

Infrastructure Cost at Scale The combination of Snowflake, managed Spark clusters, and Amazon S3 provides strong scalability guarantees, but the cost profile grows non-linearly under high ingestion volumes. Unoptimised dbt models and inefficient Spark jobs can lead to ballooning query costs and unnecessary cloud egress fees as data volume increases.

Multilingual and Multimodal Coverage The current pipeline is scoped to English-language text content only. This limits the reach of the credibility detection model as Trustra scales to broader markets. Additionally, the pipeline does not process images or embedded metadata that frequently accompany fake news articles and carry significant classification signal.

B) Future Big Data Improvements

Human-in-the-Loop Annotation Integrate a human review queue into the pipeline where low-confidence semi-supervised labels are flagged before Gold promotion. Partnerships with financial fact-checking organizations would further expand the supply of domain-specific, reliably labelled business news records over time.



Proactive Drift Detection Introduce a dedicated drift detection layer into the Airflow DAG using statistical tests such as the Population Stability Index (PSI) or the Kolmogorov-Smirnov test on incoming feature distributions. Drift detected above a configurable threshold should trigger an immediate out-of-cycle retraining run rather than waiting for the next scheduled window.

Low-Latency Streaming Migrate latency-sensitive pipeline stages to Apache Flink, which offers true event-driven streaming with sub-second end-to-end latency, while retaining Spark for heavy batch reprocessing and historical feature engineering. Alternatively, Spark's newly introduced Real-Time Mode — targeting P99 end-to-end latencies within 500ms for Kafka-to-Kafka queries — represents a viable upgrade path within the existing technology stack.

Cost Governance Controls Implement query budgeting in Snowflake, Spark job profiling to eliminate shuffle-heavy transformations, and tiered S3 storage policies that automatically archive Bronze data older than 90 days to cold storage such as Amazon S3 Glacier. dbt model materialization strategies should be reviewed periodically to ensure incremental models are not performing full table scans under edge-case conditions.

Automated Scheduled Model Retraining Introduce a fully automated retraining pipeline that periodically extracts a refreshed training corpus directly from the Snowflake Gold layer at fixed time intervals — for example, weekly or monthly — and triggers the DistilBERT fine-tuning process without requiring any manual intervention from the admin. The Airflow DAG would orchestrate the end-to-end flow: querying the Gold layer for new validated records since the last training run, merging them with the existing corpus, launching the training job on the configured compute environment (e.g., Google Colab GPU or a managed cloud training service), evaluating the resulting model against a held-out validation set, and automatically deploying the updated model weights to the Flask API endpoint if performance meets the defined threshold. This removes the current dependency on admin-initiated retraining, ensures the model continuously adapts to evolving language patterns and emerging misinformation tactics, and closes the concept drift gap identified in the limitations above.

5.2 AI Limitations and Future Improvement

A) AI Limitations

Despite strong overall performance, the system has some inherent limitations that must be acknowledged before deployment in real-world settings:

- **Dataset Scope and Distribution Shift:** The model was trained exclusively on English-language news articles from a specific historical period. It has not been exposed to emerging misinformation formats, news from different cultural contexts, or articles in other languages. Deployment in a different news ecosystem may result in significant accuracy degradation without retraining.
- **Category Boundary Ambiguity:** Several news categories in the dataset are linguistically close in particular, Government News, left-news, and politics share overlapping vocabulary, writing style, and subject matter. The model inherits this ambiguity, resulting in lower F1 scores for those categories (as low as 29% for left-news). This reflects a labelling problem in the original data as much as a model limitation.

- **Small-Class Underperformance:** Categories with few training examples notably Middle-east (55 test samples) and US_News (58 test samples); achieve lower performance than high-frequency categories like politicsNews and worldnews. The WeightedRandomSampler partially mitigates this for the binary fake-detection head, but the category head still suffers from data imbalance.

B) Future AI Improvements

The following enhancements are planned or recommended to address the limitations identified above and push the model toward production-grade reliability:

Improvement	Technical Approach	Expected Benefit
Continual / Online Learning	Periodic retraining on recent articles using a sliding time window or replay buffer.	Keeps the model current with evolving language and new misinformation tactics.
Larger Backbone (BERT / RoBERTa)	Replace DistilBERT with RoBERTa-base or BERT-large for tasks requiring deeper semantic understanding.	Likely improves borderline categories (Government News, left-news) at the cost of inference speed.
Data Augmentation	Back-translation, paraphrasing with GPT-4, and synonym substitution for low-frequency categories.	Balances the training distribution for Middle-east, US_News, left-news, and Government News.
Explainability Layer (XAI)	Integrate SHAP or LIME to attribute prediction confidence to specific tokens or phrases.	Makes the model auditable and increases trust with editorial and legal stakeholders.
Multi-lingual Support	Replace DistilBERT with mBERT or XLM-RoBERTa to handle Arabic, French, Spanish, and other languages.	Extends detection capability to non-English-language news ecosystems.
Metadata Fusion	Incorporate non-textual signals: publication date, author history, domain reputation, social share velocity.	Provides context beyond article text, reducing false negatives on stylistically convincing fake news.

Table 20: Future AI Improvements

In summary, the multi-task DistilBERT system demonstrates state-of-the-art performance on the benchmark dataset, with ~99% accuracy on fake news detection and ~93.5% accuracy on category classification. The architecture's shared-encoder design provides efficiency and cross-task generalization. Addressing the outlined limitations particularly category data imbalance, temporal drift, and explainability will be the focus of the next development iteration.



A) Backend Limitations

- The backend currently depends on external AI services for news verification and chatbot responses, which may introduce delays, temporary unavailability, or inconsistent results.
- News verification is processed asynchronously, requiring polling from the frontend until the verification status changes from pending to either real or fake.
- JWT-based authentication currently relies on token expiration and refresh mechanisms without advanced session management features such as device tracking or multi-session control.
- Avatar images are stored as Base64 strings, which may increase database size and reduce performance compared to dedicated cloud file storage solutions.
- The system uses fixed rate-limiting rules that may affect user experience during high traffic periods or intensive AI usage.
- The chatbot conversation history is stored as JSON-based data, which may become less efficient as conversation history grows over time.
- Search functionality is currently limited to username-based searching and does not include advanced filtering or recommendation capabilities.
- The platform does not currently provide real-time communication features such as WebSockets or live notifications for news verification updates.
- The AI classification model may occasionally misclassify news articles due to limitations in natural language understanding and external data reliability.
- The backend currently lacks a dedicated analytics and monitoring dashboard for tracking system performance, user activity, and AI accuracy metrics.

B) Future Backend Improvements

- Implement real-time updates using WebSockets or Server-Sent Events (SSE) to replace polling for news verification status changes.
- Integrate cloud-based image storage services such as Cloudinary or AWS S3 instead of storing avatars as Base64 strings.
- Improve AI verification accuracy by integrating multiple AI models and fact-checking data sources.
- Add advanced authentication and security features such as email verification, two-factor authentication (2FA), device/session management, and OAuth login providers.
- Introduce caching mechanisms using technologies such as Redis to improve API response times and reduce database load.
- Develop a scalable microservices architecture to separate AI processing, authentication, and news management into independent services.
- Add real-time notification systems for likes, shares, verification completion, and chatbot interactions.
- Implement advanced search and filtering capabilities for news articles, users, and categories.
- Add logging, monitoring, and analytics tools for better performance tracking and error management.
- Improve chatbot memory handling and context management to provide more personalized and accurate AI responses.

- Introduce automated testing pipelines, CI/CD deployment workflows, and containerization using Docker for easier maintenance and scalability.
 - Add multilingual support and localization features to improve accessibility for users from different regions.
 - Enhance API security through stricter validation, request sanitization, API versioning, and improved protection against common web vulnerabilities.
-

5.4. Frontend Limitations & Future Improvements

A) Frontend Limitations

- Forgot password functionality is currently UI-only and not connected to backend services.
- Avatar upload functionality is prepared in the API layer but not implemented in the UI.
- Custom navigation routing is used instead of React Navigation.
- Some backend validation issues still affect the signup process.
- Community feed access may fail for non-authenticated users.

B) Future Frontend Improvements

- Integrate React Navigation for advanced routing and transitions.
- Add image picker functionality for profile avatars.
- Implement real OTP and password recovery systems.
- Add push notification support using Expo Notifications.
- Implement offline caching and local feed storage.
- Add dark mode support for improved accessibility.
- Implement save/bookmark and delete news features.
- Improve responsive UI for tablets and larger devices.

The Trustra frontend application demonstrates the use of modern mobile development technologies to create an AI-powered news verification platform. The project successfully integrates authentication, AI analysis, chatbot support, and responsive user interfaces into a single mobile application.



This project has successfully developed Trustra, a comprehensive AI-powered mobile application designed to combat the proliferation of fake business news while simultaneously empowering entrepreneurs and business owners with intelligent, accessible business guidance. By integrating Deep Learning, Natural Language Processing, Big Data engineering, Business Intelligence, and conversational AI into a single unified platform, Trustra addresses a meaningful gap in the current digital information landscape.

The core fake news detection capability, built on a fine-tuned DistilBERT multi-task learning model, demonstrated near-perfect performance on the held-out test set, achieving 99.6% overall accuracy and a Macro F1 score of 99.6% on the binary fake/real classification task, and approximately 93.5% accuracy on the eight-category news classification task. These results confirm that the DistilBERT architecture, chosen for its balance of classification accuracy, computational efficiency, and real-time inference suitability, was the correct model selection for a publicly deployed mobile application that must deliver immediate results to end users.

The Big Data pipeline, orchestrated by Apache Airflow and spanning Apache Kafka, Amazon S3, Apache Spark, Snowflake, and dbt, establishes a scalable, production-grade data engineering architecture. The three-layer Medallion design (Bronze, Silver, Gold) ensures data quality gates are enforced progressively, and the pipeline's horizontal scalability at every tier means that the system can accommodate growth in both user volume and data ingestion without requiring architectural redesign. The resulting analytics-ready Gold datasets power interactive Power BI dashboards that provide actionable business intelligence insights and validate the project concept through market research findings.

The Trusty AI business expert chatbot, powered by the xAI Grok API, fulfills the platform's secondary objective of providing accessible business consultation to entrepreneurs who may lack the resources to engage professional advisors. Trusty's persistent, per-user conversation history and its domain-scoped specialization in business planning, marketing strategy, financial management, feasibility studies, SWOT analysis, and startup development collectively deliver a practical and intelligent advisory experience within the mobile platform.

The trusted community feed, with its verification-before-publication workflow and interaction-restricted design (reactions only, no open comments), creates a controlled information environment that reduces the propagation of misinformation while enabling genuine community engagement with verified business content. The secure RESTful backend, deployed on Railway with JWT-based dual-token authentication, three-tier rate limiting, and a consistent uniform API envelope, provides a robust, maintainable, and scalable foundation for the application's services.

The comparative analysis conducted in Chapter 2 contextualizes Trustra's contribution relative to existing solutions. Unlike Logically, which targets institutional clients and relies on human fact-checkers, Trustra is publicly accessible and delivers automated real-time classification. Unlike NewsGuard, which evaluates news domains manually rather than individual articles, Trustra performs article-level fake news detection with



quantitative confidence scores and supports user-generated content in a moderated community environment. Trustra occupies a distinct position: automated and scalable like commercial AI systems, but community-focused and publicly accessible like consumer platforms.

While the system demonstrates strong performance, several limitations were identified and documented throughout this thesis. The DistilBERT model operates primarily on English-language content, limiting multilingual reach. Certain underrepresented news categories, such as left-news and Government News, show lower classification F1 scores due to data imbalance and semantic overlap with adjacent categories. The Big Data pipeline's micro-batch processing model introduces latency relative to true real-time streaming, and the current deployment does not incorporate human fact-checkers or an evidence retrieval mechanism, which reduces explainability for non-technical users. These limitations are clearly acknowledged and constitute the primary roadmap for future development, including multilingual model fine-tuning, data augmentation for imbalanced categories, low-latency streaming migration, and explainability layer integration via SHAP or LIME.

In conclusion, Trustra successfully demonstrates that it is technically feasible and practically valuable to unify fake news detection, AI-powered business consultation, scalable data engineering, and community-driven news sharing into a single mobile platform. The project achieves its stated objectives and contributes a meaningful, deployable solution for combating business misinformation while equipping entrepreneurs with data-driven insights and trusted knowledge. The architecture, model, and engineering decisions made throughout this project provide a solid foundation upon which future iterations can build to extend the platform's capabilities, languages, and reach.



Amazon Web Services. (n.d.). *Amazon Web Services*. <https://aws.amazon.com/>

Apache Software Foundation. (n.d.). *Apache Airflow documentation*. <https://airflow.apache.org/docs/>

Apache Software Foundation. (n.d.). *Apache Kafka documentation*. <https://kafka.apache.org/documentation/>

Apache Software Foundation. (n.d.). *Apache Spark documentation*. <https://spark.apache.org/>

bcryptjs. (2024). *bcryptjs: Optimized bcrypt in JavaScript* [npm package].
<https://www.npmjs.com/package/bcryptjs>

Expo. (n.d.). *Expo*. <https://expo.dev/>

Expo. (n.d.). *Expo Go*. <https://expo.dev/client>

Expo. (n.d.). *Expo Router documentation*. <https://docs.expo.dev/router/introduction/>

Express.js. (2024). *Express: Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>

express-rate-limit. (2024). *express-rate-limit: Basic rate-limiting middleware for Express* [npm package].
<https://www.npmjs.com/package/express-rate-limit>

express-validator. (2024). *express-validator documentation*. <https://express-validator.github.io/docs/>

Groq. (2024). *Groq API documentation*. <https://console.groq.com/docs>

jsonwebtoken. (2024). *jsonwebtoken: JSON Web Token implementation for Node.js* [npm package].
<https://www.npmjs.com/package/jsonwebtoken>

Logically. (n.d.). *Logically*. <https://www.logically.ai>

Microsoft. (n.d.). *Visual Studio Code*. <https://code.visualstudio.com>

MySQL. (2024). *MySQL 8.0 Reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

NewsGuard Technologies. (n.d.). *NewsGuard*. <https://www.newsguardtech.com>

Ngrok. (n.d.). *Ngrok*. <https://ngrok.com>

Node.js. (2024). *Node.js documentation*. <https://nodejs.org/en/docs>

npm, Inc. (n.d.). *npm*. <https://npmjs.com>

Railway. (2024). *Railway documentation*. <https://docs.railway.app>

React Native. (n.d.). *Components and APIs*. <https://reactnative.dev/docs/components-and-apis>

React Native. (n.d.). *FlatList*. <https://reactnative.dev/docs/flatlist>

React Native. (n.d.). *KeyboardAvoidingView*. <https://reactnative.dev/docs/keyboardavoidingview>

React Native. (n.d.). *React Native*. <https://reactnative.dev>

React Native Community. (n.d.). *React Native DateTimePicker*. <https://github.com/react-native-datetimepicker/datetimepicker>

React Native Community. (n.d.). *AsyncStorage documentation*. <https://react-native-async-storage.github.io/async-storage/>

REST API Tutorial. (n.d.). *What is REST? RESTful API tutorial*. <https://restfulapi.net>

Sequelize. (2024). *Sequelize v6 documentation*. <https://sequelize.org/docs/v6/>

Snowflake Inc. (n.d.). *Snowflake*. <https://www.snowflake.com/en/>

JWT.IO. (n.d.). *JSON Web Tokens*. <https://jwt.io>